



1D convolutional neural networks and applications: A survey

Serkan Kiranyaz^{a,*}, Onur Avci^b, Osama Abdeljaber^c, Turker Ince^d,
Moncef Gabbouj^e, Daniel J. Inman^f

^a Department of Electrical Engineering, Qatar University, Qatar

^b Civil, Construction and Environmental Engineering, Iowa State University, Ames, IA, USA

^c Department of Building Technology, Linnaeus University, Växjö, Sweden

^d Electrical & Electronics Engineering Department, Izmir University of Economics, Turkey

^e Department of Computing Sciences, Tampere University, Finland

^f Department of Aerospace Engineering, University of Michigan, Ann Arbor, MI, USA

ARTICLE INFO

Article history:

Received 20 March 2020

Received in revised form 13 July 2020

Accepted 28 October 2020

Available online 14 November 2020

Keywords:

Artificial Neural Networks

Machine learning

Deep learning

Convolutional neural networks

Structural health monitoring

Condition monitoring

Arrhythmia detection and identification

Fault detection

Structural damage detection

ABSTRACT

During the last decade, Convolutional Neural Networks (CNNs) have become the *de facto* standard for various Computer Vision and Machine Learning operations. CNNs are feed-forward Artificial Neural Networks (ANNs) with alternating convolutional and subsampling layers. Deep 2D CNNs with many hidden layers and millions of parameters have the ability to learn complex objects and patterns providing that they can be trained on a massive size visual database with ground-truth labels. With a proper training, this unique ability makes them the primary tool for various engineering applications for 2D signals such as images and video frames. Yet, this may not be a viable option in numerous applications over 1D signals especially when the training data is scarce or application specific. To address this issue, 1D CNNs have recently been proposed and immediately achieved the state-of-the-art performance levels in several applications such as personalized biomedical data classification and early diagnosis, structural health monitoring, anomaly detection and identification in power electronics and electrical motor fault detection. Another major advantage is that a real-time and low-cost hardware implementation is feasible due to the simple and compact configuration of 1D CNNs that perform only 1D convolutions (scalar multiplications and additions). This paper presents a comprehensive review of the general architecture and principals of 1D CNNs along with their major engineering applications, especially focused on the recent progress in this field. Their state-of-the-art performance is highlighted concluding with their unique properties. The benchmark datasets and the principal 1D CNN software used in those applications are also publicly shared in a dedicated website. While there has not been a paper on the review of 1D CNNs and its applications in the literature, this paper fulfills this gap.

© 2020 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Artificial neurons used in conventional ANNs are the first-order (linear) models of biological neurons. In the mammalian nervous system, the biological learning is mainly performed at the cellular level. As shown in Fig. 1, each neuron is capable of

* Corresponding author.

E-mail addresses: mkiranyaz@qu.edu.qa (S. Kiranyaz), oavci@iastate.edu (O. Avci), osama.abdeljaber@lnu.se (O. Abdeljaber), turker.ince@izmirkekonimi.edu.tr (T. Ince), moncef.gabbouj@tuni.fi (M. Gabbouj), daninman@umich.edu (D.J. Inman).

<https://doi.org/10.1016/j.ymssp.2020.107398>

0888-3270/© 2020 The Author(s). Published by Elsevier Ltd.

This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

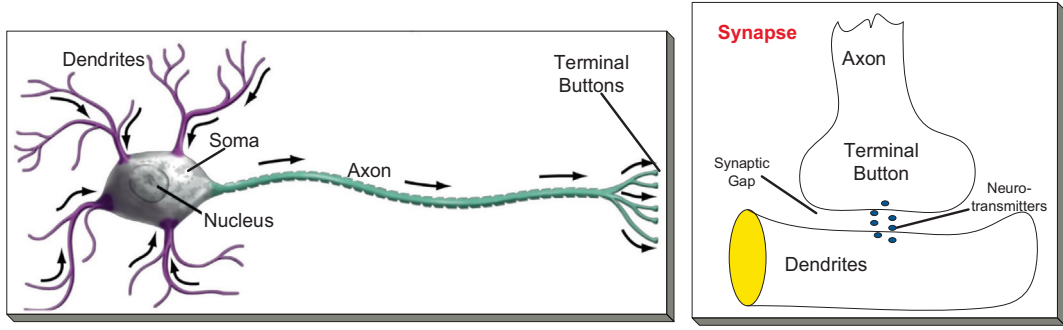


Fig. 1. A biological neuron (left) with the direction of the signal flow and a synapse (right) [7].

processing the electrical signal based on the three individual operations [1,2]: 1) reception of the other neurons outputs through the synaptic connections in *Dendrites*, 2) the integration (or pooling) of the processed output signals in the *soma* at the nucleus of the cell, and, 3) the activation of the final signal at the first part of the *Axon* or the so-called *Axon hillock*: if the pooled potentials exceed a certain limit, it “activates” a series of pulses (action potentials). As shown in Fig. 1(b), each terminal button is connected to other neurons across a small gap called a *synapse*. During the 1940s the first “artificial neuron” model was proposed by McCulloch-Pitts [3], which has thereafter been used in various feed-forward ANNs such as Multi-Layer Perceptrons (MLPs). As expressed in Eq. (1), in this popular model the artificial neuron performs a linear transformation through a weighted summation by the scalar weights. So, the basic operations performed in a biological neuron, that operate the individual synaptic connections with specific neurochemical operations and the integration in the cell’s *soma* are modeled as the linear transformation (linear weighted sum) followed by a possibly nonlinear thresholding function, $f(\cdot)$, which is called activation function.

$$x_k^l = b_k^l + \sum_{i=1}^{N_{l-1}} w_{ik}^{l-1} y_i^{l-1} \quad \text{and} \quad y_k^l = f(x_k^l) \quad (1)$$

The concept of “Perceptron”, was proposed by Frank Rosenblatt in his seminal work [4]. When used in all neurons of a MLP, this linear model is a basic model of the biological neurons leading to well-known variations in learning and generalization performances for various problems [4–8]. In the literature, there have been some attempts to change MLPs by modifying the neuron model and/or the conventional Back Propagation (BP) algorithm [9–11], or the MLP configuration [12–14] or even the way to update the network parameters (weights and biases) [15]. The most promising variant is called Generalized Operational Perceptrons [7,8], which is a heterogeneous network with non-linear operators and has thus exhibited significantly superior performance than MLPs; however, this is still the most common network model that has inspired the modern-age ANNs that are being used today.

Starting from the 1959, Hubel and Wiesel have established the foundations of the visual neuroscience through the study of the visual cortical system of cats. Their collaboration has lasted more than 25 years during which they have described the major responsive properties of the visual cortical neurons, the concept of receptive field, the functional properties of the visual cortex and the role of the visual experience in shaping the cortical architecture, in a series of articles published in The Journal of Physiology [16–20]. They are the pioneers who found the hierarchical processing mechanism of information in the visual cortical pathway, which eventually led to the Nobel Prize in Physiology or Medicine in 1981. With these advances in neurocognitive science, Fukushima and Miyake [21] in 1982 proposed the predecessor of Convolutional Neural Networks (CNNs), at the time called as “Neocognitron” which is a self-organized, hierarchical network and has the capability to recognize stimulus patterns based on the differences in their appearances (e.g., shapes). This was the first network, which has the unique ability of a biological mammalian visual system, that is, the assessment of similar objects to be assigned to the same object category independent from their position and certain morphological variations. However, in an attempt to maximize the learning performance, the crucial need of a supervised method to train (or adapt) the network for the learning task in hand became imminent. The ground-breaking invention of the Back-Propagation (BP) by Rumelhart and Hinton in 1986 [22] became a major cornerstone of the Machine Learning (ML) era. BP incrementally optimizes the network parameters, i.e., weights and biases, in an iterative manner using the gradient descent optimization technique.

These two accomplishments have started a new wave of approaches that eventually created the first naïve CNN models but it was the seminal work of Yann LeCun in 1990 who formulated the BP to train the first CNN [23], the so-called “LeNet”. This CNN ancestor became mature in 1998 and its superior classification power was demonstrated in [24] over the benchmark MNIST handwritten number database [25]. This success has begun the era of CNNs and brought a new hope to otherwise “idle” world of ML during the 1980s and early 90s. CNNs have been used in many applications during the 90s and the first decade of the 21st century but soon they fell out of fashion especially with the emergence of new generation ML paradigms such as Support Vector Machines (SVMs) and Bayesian Networks (BNs). There are two main reasons for this. First, small or medium size databases were insufficient to train a deep CNN with a superior generalization capability. Then of

course, training a deep CNN is computationally very demanding and feasible only with the modern graphical processors present today. This is why during these two decades the application of CNNs has been limited only to low-resolution (e.g. the thumbnail size) and gray-scale images in small-size datasets. On the contrary, both SVMs and BNs in comparison have fewer parameters that can be well optimized especially over such small to medium size datasets and independent from the image resolution. The annual ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in the image classification competition in 2012 became the turning point for the application of deep CNNs in the area of large-scale image classification. For this competition, Krizhevsky et al. proposed the deep CNN model for the first time, the so-called AlexNet [26] which is the ancestor of the Deep Learning paradigm. AlexNet was an 8-layer CNN (5 convolutional-pooling layers and 3 fully-connected layers) that achieved 16.4% error rate in the ImageNet benchmark database [27] and this was about 10% lower than the second top method that uses a traditional ML approach, i.e., the Support Vector Machine (SVM) classifier network over the traditional visual features of Scale Invariant Feature Transform (SIFT) [28] and Local Binary Patterns (LBP) [29]. The ImageNet database contains more than one million images for training and divided into 1000 visual categories. The same study [26] also proposed some novel architectural features such as Rectified Linear Units (ReLU) instead of traditional activation functions such as Sigmoids (*sigm*) or Tangent Hyperbolics (*tanh*). The AlexNet team also proposed the dropout technique in [30] to improve the generalization capability of the network. However, the most important factor which made CNNs the mainstream method afterwards was the ability to train them over a massive size dataset by using parallelized computational paradigms over the emerging graphical processing units (GPUs).

With the successful introduction of AlexNet, the era of deep 2D CNNs has begun and immediately replaced the traditional classification and recognition methods within a short time. Deep CNNs eventually have become the primary tool used in any Deep Learning (DL) application including the contemporary ILSVRC image classification competitions. The following year, a new network, the so-called ZFnet [31] was proposed by Zeiler and Fergus that became the winning CNN model of the ILSVRC 2013. ZFnet further reduced the error rate down to 11.7% on the ImageNet database. The authors have shown how to visualize each convolution layer of the CNN which in turn has deepened our understanding why CNNs achieve such superior discrimination power among different visual object categories. The following year in ILSVRC 2014, a new breakthrough was achieved by the Google team with a deeper CNN, called as “GoogLeNet” with a codename “Inception”, which almost halved the best error rate down to 6.7% in the ImageNet database. GoogLeNet has been designed by increasing the depth (with a 22 convolutional layers) and also the width of the network while keeping the computational budget constant. Besides using a deeper network with sparse connections, the key idea is that GoogLeNet obtained the top object recognition performance in ILSVRC 2014 with an ensemble of 6 CNNs. Since then, the popularity of the deep CNNs has peaked and eventually they became the *de facto* standard for various ML and computer vision applications over the years. Furthermore, they have been frequently used in processing sequential data including Natural Language Processing and Speech Recognition [32,33] and even 1D signals e.g., vibration [34,35].

Besides the top performance levels they can achieve, another crucial advantage they offer is that they can combine both feature extraction and classification tasks into a single body unlike traditional Artificial Neural Networks (ANNs). While conventional Machine Learning (ML) methods usually perform certain pre-processing steps and then use fixed and hand-crafted features which are not only sub-optimal but may usually require a high computational complexity, CNN-based methods can extract the “learned” features directly from the raw data of the problem at hand to maximize the classification accuracy. This is indeed the key characteristic for improving the classification performance significantly which made CNNs attractive to complicated engineering applications. However, the reign of traditional ML approaches was still unchallenged for 1D signals since deep CNNs were modeled and created specifically for 2D signals and their application was not straightforward for 1D signal signals especially when the data is scarce. The direct utilization of a deep CNN for a 1-D signal processing application naturally needs a proper 1D to 2D conversion. Recently, researchers have tried to use deep CNNs for fault diagnosis of bearings [34–42]. For this purpose, different conversion techniques have been utilized to represent the 1D vibration signals in 2D. A commonly used technique is to directly reshape the vibration signal into an $n \times m$ matrix called “the vibration image” [39]. Another technique was used in [37] where two vibration signals were measured using two accelerometers. Then, Discrete Fourier Transform (DFT) was applied, and the two transformed signals were represented in a matrix which can be fed into a conventional deep CNN. For electrocardiogram (ECG) beat classification and arrhythmia detection, the common approach is to first compute power- or log-spectrogram to convert each ECG beat to a 2D image [43,44]. However, there are certain drawbacks and limitations of using such deep CNNs. Primarily, it is known that they pose a high computational complexity which requires special hardware especially for training. Therefore, 2D CNNs are not suitable for real-time applications on mobile and low-power/low-memory devices. In addition, proper training of deep CNNs requires a massive size dataset for training to achieve a reasonable generalization capability. This may not be a viable option for many practical 1D signal applications where labeled data can be scarce.

To incorporate such drawbacks, in 2015 Kiranyaz et al. [45] proposed the first compact and adaptive 1D CNNs to operate directly on patient-specific ECG signals. In a relatively short time, 1D CNNs have become popular with a state-of-the-art performance in various signal processing applications such as early arrhythmia detection in electrocardiogram (ECG) beats [45–47], structural health monitoring and structural damage detection [48–52], high power engine fault monitoring [53] and real-time monitoring of high-power circuitry [54]. Furthermore, two recent studies have utilized 1D CNNs for damage detection in bearings [55–58]. However, in the latter study conducted by Zhang et al. [58], both single and ensemble of deep 1D CNN(s) were created to detect, localize, and quantify bearing faults. A deep configuration of 1D CNN used in this study consisted of 6 large convolutional layers followed by two fully connected (dense) layers. Other deep 1D CNN approaches

have been recently proposed by [59–62] for anomaly detection in ECG signals. These deep configurations share the common drawbacks of their 2D counterparts. For example, in [58], several “tricks” were utilized to improve the generalization performance of the deep 1D CNN such as data augmentation, batch normalization, dropout, majority voting, etc. Another approach to tackle this problem is to utilize the majority of the dataset for training which may not be feasible in some practical applications. In the study [58], more than 96% of the total data is used to train the deep network. With that, the assumption that such a large set of training data would be available may hinder the utilization of this method in practice. Therefore, in this article the focus is drawn particularly on compact 1D CNNs with few hidden layers/neurons, and their applications to some major engineering problems with the assumption that the labeled data is scarce, or application or device-specific solutions are required to maximize the detection and identification accuracy. The benchmark datasets and the principal 1D CNN software used in those applications are now publicly shared in [63].

The motivation for this survey is to offer a comprehensive overview of 1D CNNs, both theoretically, and from an application and a methodology driven perspective. This survey includes over 90 papers, most of them recent, on a wide variety of applications of conventional 2D CNNs and of course, the recent 1D variants. When overlapping work had been reported in multiple publications, only the publication(s) deemed most important were included. We expect the search terms used to cover most of the work incorporating compact 1D CNNs and its deep variants. The state-of-the-art 1D CNN applications on real-time electrocardiogram monitoring and anomaly detection were presented in detail in Appendix A. Finally, we leveraged our own experience with the application of compact 1D CNNs to various application domains to provide readers a detailed insight covering the state-of-the-art, some of the current open challenges and overview of research directions which we think that they will become important in the near future.

The rest of the paper is organized as follows. Section 2 provides a general background on adaptive and compact 1D CNNs with the formulation for Back-Propagation (BP) training. Section 3 presents a brief review on popular engineering applications of the 1D CNNs. Section 4 presents a detailed computational complexity analysis of the 1D CNNs and the computational times of the competing methods on a sample application domain. Finally, Section 5 concludes the paper and suggests topics for future directions on 1D CNNs.

2. Overview of convolutional neural networks

Deep Learning (DL) is the latest achievement of the Machine Learning era where it has presented near-human initially, and nowadays super-human abilities in many applications including voice-to-text translations, object detection and recognition, anomaly detection, recognizing emotions from audio or video recordings, etc. Even before the introduction of the AlexNet, perhaps one can consider that this era has begun with the ground-breaking article published in the journal, *Science*, in 2006 by Hinton and Salakhutdinov [59], which explained the role of “the depth” of an ANN in machine learning. It basically points out the fact that ANNs with several hidden layers can have a powerful learning ability, which can further be improved with the increasing depth –or equivalently the number of hidden layers. Hence comes the term “Deep” learning, a particular ML branch, which can tackle complex patterns and objects in massive size datasets.

In this section, we shall begin with the fundamental tool of DL, the deep (and conventional) CNNs whilst explaining their basic features and blocks. We will briefly discuss the most popular deep CNNs ever proposed and then move on with the most recent CNN architecture, the 1D CNNs, which are focused solely on 1D signal and data repositories. The particular focus will be drawn on compact and adaptive 1D CNN models, which can promise certain advantages and superiorities over their deep 2D counterparts.

2.1. 2D convolutional neural networks

Although it has been almost 30 years after the first CNN was proposed, modern CNN architectures still share the common properties with the very first one such as convolutional and pooling layers. Also, besides few variations, the popular training method, the Back-Propagation technique is another commonality since 90s. This section will provide a brief overview of the conventional deep CNNs while introducing the most fundamental ideas and cornerstone architectures of the past.

To start with, the popularity and the wide range of application domains of deep CNNs can be attributed to the following advantages:

1. CNNs fuse the feature extraction and feature classification processes into a single learning body. They can learn to optimize the features during the training phase directly from the raw input.
2. Since CNN neurons are sparsely-connected with tied weights, CNNs can process large inputs with a great computational efficiency compared to the conventional fully-connected Multi-Layer Perceptrons (MLP) networks.
3. CNNs are immune to small transformations in the input data including translation, scaling, skewing and distortion.
4. CNNs can adapt to different input sizes.

In a conventional MLP, each hidden neuron contains scalar weights, input and output. However, due to the 2D nature of images, each neuron in a CNN contain 2-D planes for weights, which is known as the kernel, and input and output which is known as the feature map. Fig. 2 illustrates the basic blocks of a sample CNN configuration that classifies a 24×24 -pixel

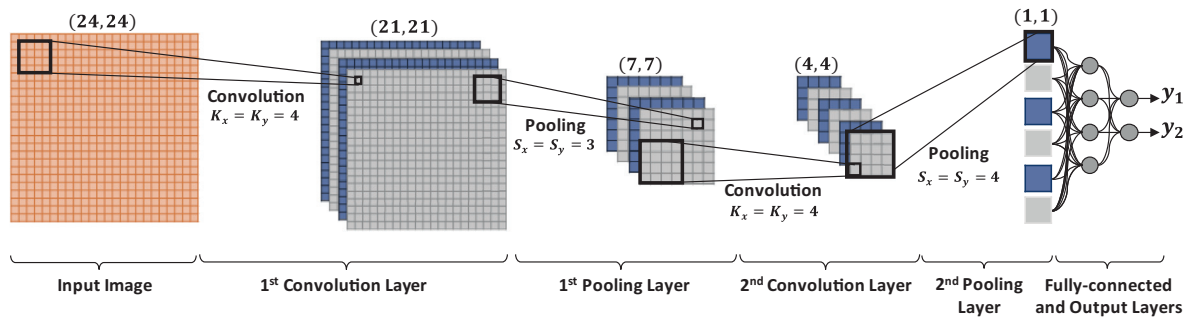


Fig. 2. The illustration of a sample CNN with 2 convolution and one fully-connected layers.

grayscale image into two categories. This sample network consists of two convolution and two pooling layers with 4 and 6 neurons, respectively. The output of the last pooling layer is processed by a single fully-connected layer and followed by the output layer that produces the classification output. The interconnections feeding the convolutional layers are assigned by weighting filters (w) having a kernel size of (K_x, K_y) . The convolution takes place within the image boundaries; therefore, the feature map dimension is reduced by the $(K_x - 1, K_y - 1)$ pixels from the width and height, respectively. The subsampling factors (S_x, S_y) are set in advance in the pooling layers. In the sample illustration in the figure, the kernel sizes corresponding to the two convolution layers were set to $K_x = K_y = 4$, while the subsampling factors are set as $S_x = S_y = 3$ for the first pooling layer and $S_x = S_y = 4$ for the second one. Note that these values were deliberately selected so that the outputs of the last pooling layer (i.e. the input to the fully-connected layer) are scalars. (1x1). The output layer consists of two fully-connected neurons corresponding to the number of classes to which the image is categorized. The following steps describe a complete forward-propagation process in this sample CNN:

1. A 24×24 -pixel grayscale image is fed to the input layer of the CNN.
2. Each neuron of the first convolution layer performs a linear convolution between the image and corresponding filter to generate the input feature map of the neuron.
3. The input feature map of each neuron is passed through the activation function to generate the output feature map of the neuron of the convolution neuron.
4. In the pooling layer, each neuron's feature map is created by decimating the output feature map of the previous neuron of the convolution layer. In this example, 7×7 feature maps are created in the first pooling layer.
5. Steps 3 and 4 are repeated and the outputs of the second pooling layer become the inputs of the fully-connected layers, which are identical to the layers of a conventional MLP.
6. The scalar outputs are forward-propagated through the following fully-connected and output layers to produce the final output the represents the classification of input image.

CNNs are predominantly trained in a supervised manner by the so-called backpropagation (BP) algorithm. During each iteration of the BP, the gradient magnitude (or sensitivity) of each network parameter such the weights of the convolution and fully-connected layers is computed. The parameter sensitivities are then used to iteratively update the CNN parameters until a certain stopping criterion is achieved. There are several gradient-descent optimization methods that can be used in BP

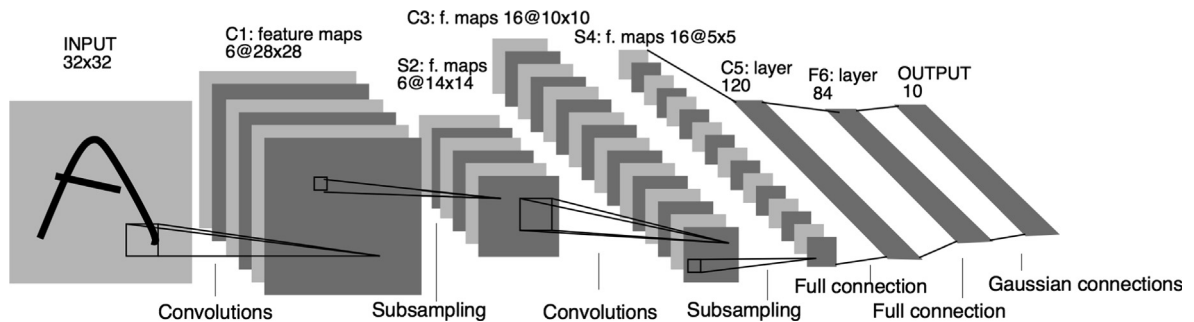


Fig. 3. The configuration of the ancestor of CNNs, the “LeNet”. The figure is taken from [24]. There are two interleaved convolutional and pooling layers following by three (two hidden and one output) fully-connected layers. The output layer is composed of 10 Radial Basis Function (RBF) neurons each of which computes the Euclidean distance between the network output and ground truth label for 10 classes.

such as Stochastic Gradient Descent (SGD), SGD with momentum [64], AdaGrad [65], RMSProp [66], Adam [67] and its variants [68]. A detailed description of the BP in 2D CNNs can be found in [46].

The configurations of the two most popular CNNs, the ancestor CNN, “LeNet” [24] and the first deep CNN, “AlexNet” [26] are shown in Fig. 3 and Fig. 4, respectively. Despite the long time between them, the conceptual and architectural similarities are obvious. Perhaps the most striking difference is the “deep” configuration of the AlexNet, which encapsulates millions of network parameters. The fundamental two properties of the convolutional layers, “weight sharing” and “limited connectivity” exist even in the most-recent CNN architectures. In fact, these are the main features which separate CNNs from the conventional MLPs; otherwise, both networks are homogenous and share the common linear neuron model.

2.2. 1D convolutional neural networks

The conventional deep CNNs presented in the previous section are designed to operate exclusively on 2D data such as images and videos. This is why they are often referred to as, “2D CNNs”. As an alternative, a modified version of 2D CNNs called 1D Convolutional Neural Networks (1D CNNs) have recently been developed [45–54]. These studies have shown that for certain applications 1D CNNs are advantageous and thus preferable to their 2D counterparts in dealing with 1D signals due to the following reasons:

- There is a significant difference in terms of computational complexities of 1D and 2D convolutions, i.e., an image with $N \times N$ dimensions convolve with $K \times K$ kernel will have a computational complexity $\sim O(N^2 K^2)$ while in the corresponding 1D convolution (with the same dimensions, N and K) this is $\sim O(NK)$. This means that under equivalent conditions (same configuration, network and hyper parameters) the computational complexity of a 1D CNN is significantly lower than the 2D CNN.
- As a general observation especially over the recent studies most of the 1D CNN applications have used compact (with 1–2 hidden CNN layers) configurations with networks having <10 K parameters whereas almost all 2D CNN applications have used “deep” architectures with more than 1 M (usually above 10 M) parameters. Obviously, networks with shallow architectures are much easier to train and implement.
- Usually, training deep 2D CNNs requires special hardware setup (e.g. Cloud computing or GPU farms). On the other hand, any CPU implementation over a standard computer is feasible and relatively fast for training compact 1D CNNs with few hidden layers (e.g. 2 or less) and neurons (e.g. < 50).
- Due to their low computational requirements, compact 1D CNNs are well-suited for real-time and low-cost applications especially on mobile or hand-held devices [45–57].

In the aforementioned recent studies, compact 1D CNNs have demonstrated a superior performance on those applications which have a limited labeled data and high signal variations acquired from different sources (i.e., patient ECG, civil, mechanical or aerospace structures, high-power circuitry, power engines or motors, etc.). As illustrated in Fig. 5, two distinct layer types are proposed in 1D CNNs: 1) the so-called “CNN-layers” where both 1D convolutions, activation function and sub-sampling (pooling) occur, and 2) Fully-connected (dense) layers that are identical to the layers of a typical Multi-layer Perceptron (MLP) and therefore called as “MLP-layers”. The configuration of a 1D-CNN is formed by the following hyper-parameters:

- 1) Number of hidden CNN and MLP layers/neurons (in the sample 1D CNN shown in Fig. 5, there are 3 and 2 hidden CNN and MLP layers, respectively).

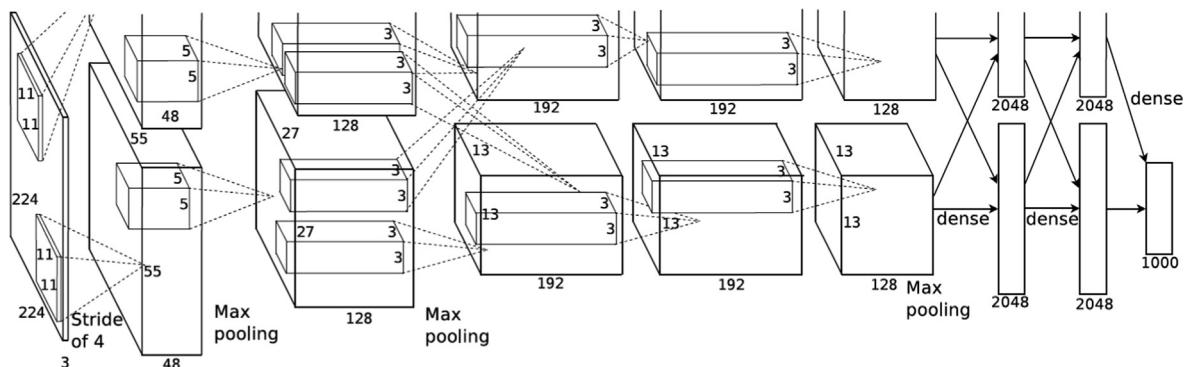


Fig. 4. The configuration of the first deep CNN, the “AlexNet” [26]. There are 5 convolutional layers and 3 max-pooling layers following by three (two hidden and one output) fully-connected (dense) layers. The numbers of both convolution and fully-connected layers are significantly higher than the LeNet. The neurons at the output layer use softmax loss of the network predictions for 1000 classes.

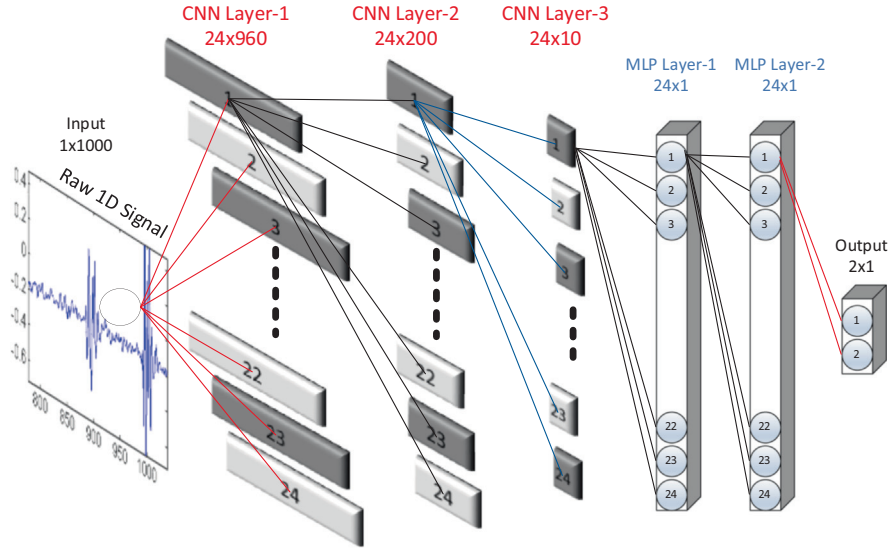


Fig. 5. A sample 1D CNN configuration with 3 CNN and 2 MLP layers.

- 2) Filter (kernel) size in each CNN layer (in the sample 1D CNN shown in Fig. 5, filter size is 41 in all hidden CNN layers).
- 3) Subsampling factor in each CNN layer (in the sample 1D CNN shown in Fig. 5, subsampling factor is 4).
- 4) The choice of pooling and activation functions.

As in the conventional 2D CNNs, the input layer is a passive layer that receives the raw 1D signal and the output layer is a MLP layer with the number of neurons equal to the number of classes. Three consecutive CNN layers of a 1D CNN are presented in Fig. 6. As shown in this figure, the 1D filter kernels have size 3 and the sub-sampling factor is 2 where the k^{th} neuron in the hidden CNN layer, l , first performs a sequence of convolutions, the sum of which is passed through the activation function, f , followed by the sub-sampling operation. This is indeed the main difference between 1D and 2D CNNs, where 1D arrays replace 2D matrices for both kernels and feature maps. As a next step, the CNN layers process the raw 1D data and “learn to extract” such features which are used in the classification task performed by the MLP layers. As a consequence, both feature extraction and classification operations are *fused* into one process that can be optimized to maximize the classification performance. This is the major advantage of 1D CNNs which can also result in a low computational complexity since the only operation with a significant cost is a sequence of 1D convolutions which are simply linear weighted sums

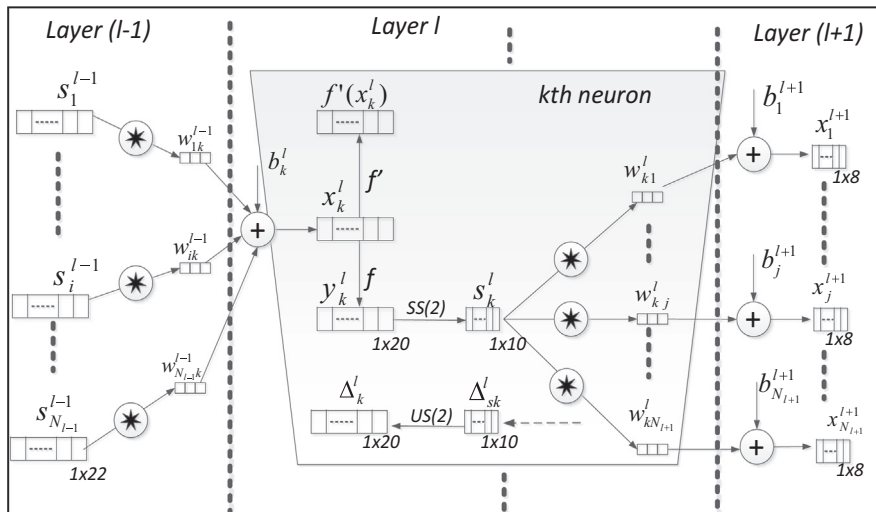


Fig. 6. Three consecutive hidden CNN layers of a 1D CNN [54].

of two 1D arrays. Such a linear operation during the Forward and Back-Propagation operations can effectively be executed in parallel.

This is also an adaptive implementation since the CNN topology will allow the variations in the input layer dimension in such a way that the sub-sampling factor of the output CNN layer is tuned adaptively. The details related to Forward and Back-Propagation in CNN layers are presented in the next sub-section.

2.3. Forward- and back-propagation in CNN-layers

In each CNN-layer, 1D forward propagation (1D-FP) is expressed as follows:

$$x_k^l = b_k^l + \sum_{i=1}^{N_{l-1}} \text{conv1D}(w_{ik}^{l-1}, s_i^{l-1}) \quad (2)$$

where x_k^l is defined as the input, b_k^l is defined as the bias of the k^{th} neuron at layer l , s_i^{l-1} is the output of the i^{th} neuron at layer $l-1$, w_{ik}^{l-1} is the kernel from the i^{th} neuron at layer $l-1$ to the k^{th} neuron at layer l . $\text{conv1D}(\cdot, \cdot)$ is used to perform 'in-valid' 1D convolution without zero-padding. Therefore, the dimension of the input array, x_k^l , is less than the dimension of the output arrays, s_i^{l-1} . The intermediate output, y_k^l , can be expressed by passing the input x_k^l through the activation function, $f(\cdot)$, as,

$$y_k^l = f(x_k^l) \text{ and } s_k^l = y_k^l \downarrow ss \quad (3)$$

where s_k^l stands for the output of the k^{th} neuron of the layer, l , and " $\downarrow ss$ " represents the down-sampling operation with a scalar factor, ss .

The back-propagation (BP) algorithm can be summarized as follows. Back propagating the error starts from the output MLP-layer. Assume $l=1$ for the input layer and $l=L$ for the output layer. Let N_l be the number of classes in the database; then, for an input vector p , and its target and output vectors, $\mathbf{t}^p$ and $[y_1^l, \dots, y_{N_l}^l]'$, respectively. With that, in the output layer for the input p ; the mean-squared error (MSE), E_p , can be expressed as follows:

$$E_p = \text{MSE}(\mathbf{t}^p, [y_1^l, \dots, y_{N_l}^l]') = \sum_{i=1}^{N_l} (y_i^l - t_i^p)^2 \quad (4)$$

To find the derivative of E_p by each network parameter, the delta error, $\Delta_k^l = \frac{\partial E}{\partial x_k^l}$ should be computed. Specifically, for updating the bias of that neuron and all weights of the neurons in the preceding layer, one can use the chain-rule of derivatives as,

$$\frac{\partial E}{\partial w_{ik}^{l-1}} = \Delta_k^l y_i^{l-1} \text{ and } \frac{\partial E}{\partial b_k^l} = \Delta_k^l \quad (5)$$

So, from the first MLP layer to the last CNN layer, the regular (scalar) BP is simply performed as,

$$\frac{\partial E}{\partial s_k^l} = \Delta s_k^l = \sum_{i=1}^{N_{l+1}} \frac{\partial E}{\partial x_i^{l+1}} \frac{\partial x_i^{l+1}}{\partial s_k^l} = \sum_{i=1}^{N_{l+1}} \Delta_i^{l+1} w_{ki}^{l+1} \quad (6)$$

Once the first BP is performed from the next layer, $l+1$, to the current layer, l , then one can carry on the BP to the input delta of the CNN layer l , Δ_k^l . Let zero order up-sampled map be: $\text{us}_k^l = \text{up}(s_k^l)$, then the delta error can be expressed as follows:

$$\Delta_k^l = \frac{\partial E}{\partial y_k^l} \frac{\partial y_k^l}{\partial x_k^l} = \frac{\partial E}{\partial \text{us}_k^l} \frac{\partial \text{us}_k^l}{\partial y_k^l} f'(x_k^l) = \text{up}(\Delta s_k^l) \beta f'(x_k^l) \quad (7)$$

where $\beta = (ss)^{-1}$. Then, the BP of the delta error $(\Delta s_k^l \xleftarrow{\beta} \Delta_i^{l+1})$ can be expressed as,

$$\Delta s_k^l = \sum_{i=1}^{N_{l+1}} \text{conv1Dz}(\Delta_i^{l+1}, \text{rev}(w_{ki}^{l+1})) \quad (8)$$

where $\text{rev}(\cdot)$ is used to reverse the array and $\text{conv1Dz}(\cdot, \cdot)$ is used to perform full 1D convolution with zero-padding. The weight and bias sensitivities can be expressed as follows:

$$\frac{\partial E}{\partial w_{ik}^l} = \text{conv1D}(\Delta s_k^l, \Delta_i^{l+1}) \text{ and } \frac{\partial E}{\partial b_k^l} = \sum_n \Delta_k^l(n) \quad (9)$$

When the weight and bias sensitivities are computed, they can be used to update biases and weights with the learning factor, ε , as,

$$w_{ik}^{l-1}(t+1) = w_{ik}^{l-1}(t) - \varepsilon \frac{\partial E}{\partial w_{ik}^{l-1}} \quad \text{and} \quad b_k^l(t+1) = b_k^l(t) - \varepsilon \frac{\partial E}{\partial b_k^l} \quad (10)$$

The forward and back-propagation in hidden CNN layers are illustrated in Fig. 7. The output sensitivity of the k^{th} neuron at the CNN layer l , Δs_k^l , is formed by back-propagating all the delta errors, Δ_i^{l+1} , at the next layer, $l+1$, by using Eq. (8), while the forward and back-propagation between the last hidden CNN layer and the first hidden MLP layer are summarized in Fig. 8. Further details of the BP algorithm are presented in [46].

Consequently, the iterative flow of the BP for the 1D raw signals in the training set can be stated as follows:

- 1) **Initialize** weights and biases (e.g., randomly, $\sim U(-0.1, 0.1)$) of the network.
- 2) For each BP iteration **DO**:
 - a. For each input sample in the dataset, **DO**:
 - i. **FP**: Forward propagate from the input layer to the output layer to find outputs of each neuron at each layer, $s_i^l, \forall i \in [1, N_l]$, and $\forall l \in [1, L]$.
 - ii. **BP**: Compute delta error at the output layer and back-propagate it to first hidden layer to compute the delta errors, $\Delta_k^l, \forall k \in [1, N_l]$, and $\forall l \in [1, L]$.
 - iii. **PP**: Post-process to compute the weight and bias sensitivities using Eq. (9).
 - iv. **Update**: Update the weights and biases by the (accumulation of) sensitivities scaled with the learning factor, ε using Eq. (10).

3. Applications of 1D CNNs

As discussed earlier, the widespread use of CNNs is mainly motivated by their inherent capability to fuse feature extraction and classification into a single adaptive learning body. Due to the aforementioned reasons, there are several application domains where compact 1D CNNs have now been preferred over their 2D deep counterparts. Some of the major engineering applications of 1D CNNs will be briefly presented in this section. 1D CNN applications on real-time electrocardiogram (ECG) monitoring will be covered in Appendix A in detail.

3.1. Automatic speech recognition

Real time and accurate automatic speech recognition (ASR) has been the ultimate aim for many decades since the early 20th century. The ASR's main objective is the one-to-one transcription of human speech into (written) words. Naturally, this is a challenging task since a human speech signal exhibits high level of variations among different speakers and the task gets even harder when there is a certain level of environmental noise or the speech style varies (e.g. due to the dialects of the

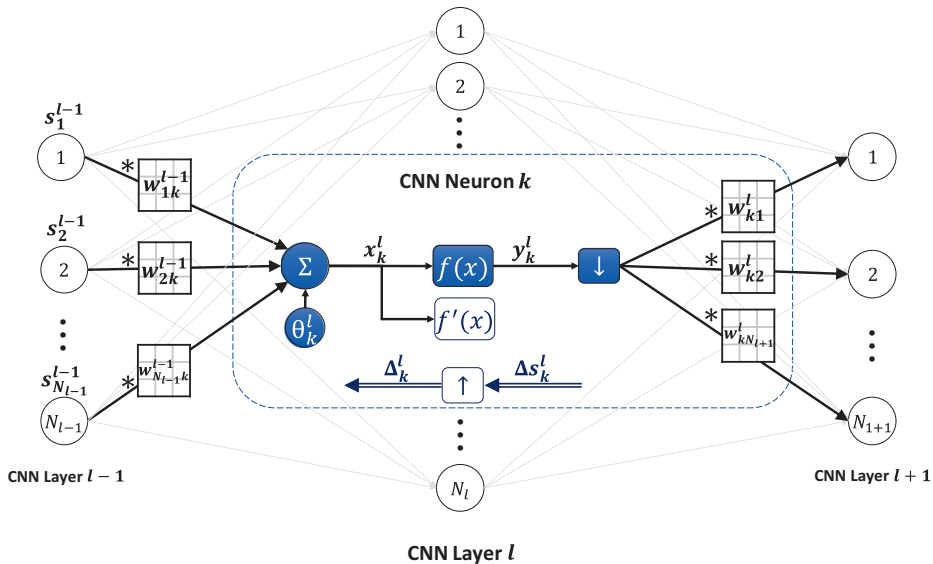


Fig. 7. Forward and back-propagation in hidden CNN layers.

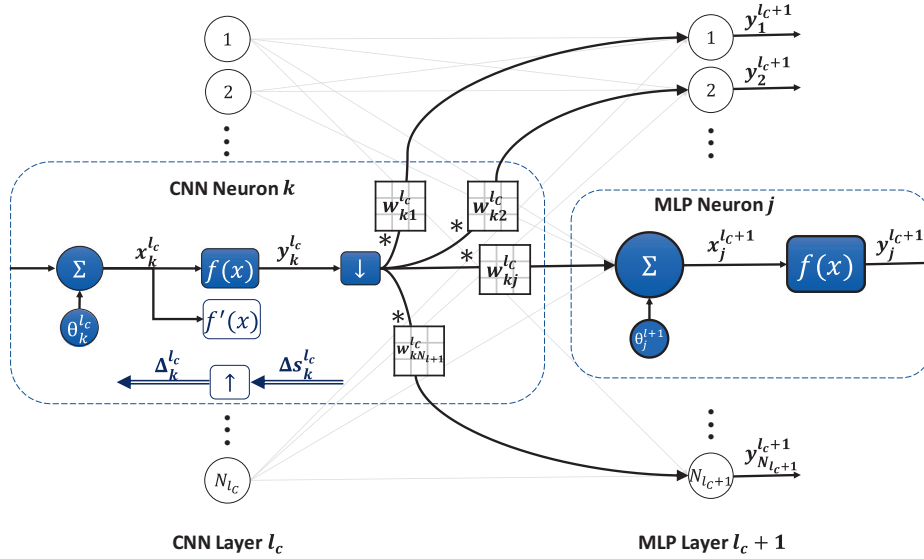


Fig. 8. Forward and back-propagation between the last hidden CNN layer and the first hidden MLP layer.

same language). Besides all, certain languages pose further challenges due to their inherent verbal structures or the speed of the spoken language and so on. Moreover, in this particular application domain, variable-length speech signals should be first mapped into structured sequences of words or phonetic symbols. Before the Deep Learning era, the most successful model to achieve this is so-called Hidden Markov Models (HMMs) that are successful of modeling the temporal behavior of speech signals using a sequence of states, each of which is associated with a particular probability distribution of observations. For each HMM state, Gaussian Mixture Models (GMMs) were primarily utilized to compute the probabilistic distribution of each phoneme (or any speech signal atom) and the fusion of GMMs-HMMs has led to many successful ASR applications providing that a discriminative training method is performed (see [69–71] for more details).

There is a particular drawback of using GMMs in HMMs: GMMs assume that the hidden expert of each sample is generated by the linear weighted sum of *only* the *Gaussian* components. In other words, there is a strict assumption about the statistical distributions of the features (they are always Gaussian). In other words, modelling the entire feature space by GMMs can pose severe limitations in the representation of the speech signal. This limitation has been overcome by a new surge of research initiatives in the Deep Learning (DL) era such as Deep Neural Networks (DNNs) and Deep Belief Networks (DBNs). Readers can refer to [72] for a detailed review. Compared to conventional GMM-HMMs, the new DNN-HMM composition has the ability to leverage highly correlated feature inputs, such as those found in much wider temporal contexts of acoustic frames, typically 9–15 frames [73].

The pioneer works where CNNs have been applied to acoustic modeling are [74,75], which performed the convolution over time windows of audio frames to classify the audio stream into generic classes such as phone, speaker and gender. The first study which applied both 1D and 2D deep CNNs to ASR is [73] where a limited-weight-sharing scheme was proposed in order to better model speech features. The proposed CNN-based approach has reduced the error rate by 6%-10% compared with DNNs on the benchmark TIMIT phone recognition dataset and some other voice search large vocabulary speech recognition tasks. This was in fact a cornerstone achievement similar to what AlexNet has accomplished in 2012 ImageNet challenge. However, it shows a major difference compared to conventional CNNs, which are applied directly over the “raw” signal. Instead, the study [73] proposed to apply log-energy computed directly from the mel-frequency spectral coefficients (i.e., with no DCT), which was denoted as MFSC features. Therefore, the input signal is the stacked MSFC features, organized as 1D feature maps. Another major difference is the “limited” weight sharing scheme which differs from the full weight sharing in conventional CNNs. The final CNNs are quite complex and deep encapsulating more than 4 M parameters overall.

Along with many successful applications of deep CNNs in image recognition, various methods based on deep CNNs [67,76–79] have been proposed and evaluated for ASR. The general approach is that time–frequency distributions (spectrograms) can first be created and then treated as “images” of a certain time duration to distinguish the individual phoneme patterns. In this way, the hidden layers of a deep CNNs can transform longer contexts in the speech and thus operates on more abstract patterns. Earlier CNN layers closer to the input layer can extract local simple patterns while latter CNN layers can detect broader and more complex patterns using the basis information available beforehand. Smaller kernels combined with more layers allow deep CNNs to exploit longer-range dependency information along both time and frequency axes more effectively.

3.2. Real-time electrocardiogram (ECG) monitoring

While cardiovascular diseases are one of the major causes of deaths on the planet, irregular (arrhythmic) heartbeats have been reported to be an indication of a cardiovascular problem. Electrocardiogram (ECG) signals are extensively used by medical practitioners to monitor and evaluate the cardiac health. During the acquisition of an ECG, the electrical activity generated by the heart muscle is measured and displayed to detect cardiac abnormality; yet, the visual inspection and analysis of ECG signals by Cardiologists is laborious, subjective and wastes the crucial time for detecting a possible cardiac threat. The automated monitoring, detection and identification of the heart signals require a real-time, accurate and robust analysis that can be achieved by a dedicated ML approach. The first 1D CNN application was on ECG beat identification [45,46] where a “patient-specific” solution was proposed, i.e., for each arrhythmia patient a dedicated compact 1D CNN was trained by using the patient-specific training data as illustrated in Fig. 11. The purpose is to identify each ECG beat into one of the five classes: N (beats originating in the sinus mode), S (supraventricular ectopic beats), V (ventricular ectopic beats), F (fusion beats), and Q (unclassifiable beats). In this study, ECG records from the benchmark MIT/BIH arrhythmia database [25] were used for both training and performance evaluation. There are total of 48 records in this benchmark database and each record has two-channel ECG signal for 30-min duration selected from 24-h recordings of 47 individuals. A total of 83,648 beats from all 44 records were used as test patterns for performance evaluation. The proposed method has achieved the highest average accuracies (99% for Ventricular Ectopic Beats (VEB) and 97.6% for Supraventricular Ectopic Beats (SVEB)) on arrhythmia detection with the minimal computational complexity.

Several studies on arrhythmia detection and identification have been proposed ever since, e.g., [46,60–62]. However, all such studies focused on ECG beat classification for cardiac patients and strictly require a certain duration of training samples (e.g. 5 min) containing both normal and abnormal beats of the patient. In the absence of abnormal beats, which is the case of a healthy person, such methods cannot be applied for the early detection of abnormal beats for an otherwise healthy person with no past history of cardiac problems. This is basically a “Chicken and Egg” problem where one needs a certain number of abnormal samples to learn their characteristics in order to discriminate them from normal beats. A recent study [47] addressed this crucial problem and proposed a “personalized” solution for the early detection of cardiac arrhythmia at the moment they appear on an otherwise “healthy” person. This became the first attempt to propose a personalized early detection of ECG anomalies and cardiac health monitoring. In the absence of real abnormal beats, this becomes a far more challenging problem than the patient-specific ECG beat classification. The key accomplishment in this work is that the common causes of cardiac arrhythmias are modeled by a set of filters and then they are used to synthesize appropriate potential abnormal beats of a healthy person as illustrated in Fig. 10. Upon learning the healthy person’s (real) normal beats and potential (synthesized) abnormal beats, the proposed system with 1D CNNs can then be used to detect any abnormal beat which may occur during monitoring. Without using the real abnormal beats in training, the proposed method has achieved accuracy level, $Acc = 80.1\%$ and false-alarm rate, $FAR = 0.43\%$. The average probability of missing the first abnormal beat, therefore, is 0.199. In addition, the average probability of missing all three consecutive abnormal beats is around 0.0079. As a result, detecting one or more abnormal beat(s) among the first three occurrences is highly probable (greater than 99.2%). The implementation details of the two major applications, [14] and [47], are presented in Appendix A.

3.3. Vibration-based structural damage detection in civil infrastructure

Monitoring of structural damage is extremely important for sustaining and preserving the service life of civil engineering structures. Meticulous and early damage detection is one of the major concerns of structural health monitoring applications in civil engineering. While successful monitoring provides resolute and staunch information on the health, serviceability, integrity and safety of structures; maintaining high performance of a structure depends highly on monitoring the occurrence, formation and propagation of damage. Damage may accumulate on structures due to different environmental and human-induced factors. Numerous monitoring and detection approaches have been developed to provide practical means for early warning against structural damage. Tremendous efforts have been put into vibration-based damage detection methods which utilize the vibration response of the monitored structure to assess its health; and identify and locate structural damage. With emerging computing power, vibration-based techniques have become feasible, proved capable and widely used for structural damage detection [97]. Researchers have very recently started to apply novel DL algorithms to develop new vibration-based damage detection techniques that do not require manual feature extraction. Recent studies have shown that both 2D and 1D CNNs achieve superior performance levels in terms of their ability to detect and locate structural damage directly from the raw vibration signals without the need for data preprocessing or extraction of hand-crafted features. Since 1D CNNs are easier to train and have lower computational complexity than their 2D counterparts, 1D CNNs are preferable when dealing with 1D vibration signals.

Conventional deep CNNs have been recently used to develop new techniques for vibration-based damage detection in civil structures. In a numerical study, Yu et al. [80] designed and trained a CNN to locate and quantify structural damage in a five-story structure. Since CNNs are only able to deal with 2D data (e.g. images), the 1D vibration signals acquired by 14 accelerometers were converted to a 2D representation simply by concatenating the 14 measured signals into a matrix. The data required for training the CNN was taken from a numerical model of the monitored structure under different damage scenarios. According to this dataset, a deep CNN having 3 CNN layers with a large number of neurons followed by 2 MLP

layers was trained using a GPU implementation. The numerical results showed that the trained CNN was successful in detecting and locating simulated structural damage.

Khodabandehlou et al. [81] developed a similar CNN-based structural damage detection technique. A one-fourth-scale laboratory structure of a reinforced concrete bridge was used to experimentally demonstrate the proposed method. This structure was used to generate vibration data corresponding to four overall damage levels ranging from “no damage” to “extreme damage”. For each damage level, the data measured by a number of accelerometers was concatenated into a single 2D matrix. The resulting dataset was used to train a deep CNN having 5 CNN and 4 MLP layers. It was demonstrated that the CNN was able to quantify the overall structural condition of the bridge directly from the measured vibration response.

In another study, Cha et al. [82] used a vision-based method with CNNs for detecting concrete cracks. It is reported that they achieved success in finding concrete cracks in realistic situations. In a similar study by Gulgec et al. [83] CNNs are used per a Python library Theano with the graphics processing unit (GPU) to classify damaged and undamaged samples modeled with Finite Element (FE) simulations only. It is reported that high classification accuracy is achieved for the FE data.

Abdeljaber et al. in [50] conducted 1D CNNs first time in vibration-based Structural Health Monitoring (SHM). A large-scale laboratory structure with plan dimensions of 5mx6m was constructed and instrumented with wired uniaxial accelerometers at Qatar University. The structure (QU grandstand simulator) is arguably the largest mock-up stadium structure built in a laboratory environment [84]. The steel structure has 30 joints at which vibrations response of the structure is recorded with accelerometers. The “damage” is introduced by simply loosening the bolts at a joint, which is an extremely slight change on the rotational stiffness as shown in Fig. 6. The vibration response of the structure under 31 damage scenarios was measured and used to train an individual 1D CNN for each sensor location. Each 1D CNN was only responsible for processing the local data measured at the corresponding location. The performance of this 1D CNN-based damage detection method was tested under a large number of single and double damage cases. A complexity analysis was also conducted to estimate the computational time required for the 1D CNNs to process the measured signals. Using an ordinary computer, when the performance of the proposed approach was tested, even with loosened bolts, all the damaged joints were detected without any misses or false alarms [50,85]. In addition, the detection speed was 45x faster than real-time speed. The 1D CNN application is optimized for multi-core CPU usage and can be obtained from [63]. This was an unprecedented achievement among all the CNN-based damage detection studies ever proposed.

In an experimental study by Avci et al. [48], the 1D CNN-based method developed in [50] was integrated with a wireless sensor network (WSN). The method was modified to allow it to analyze the signals measured by the triaxial wireless sensors. This was done so that the direction along which the damage-sensitive features are more pronounced can be determined. The modified damage detection technique was tested under a number of damage scenarios introduced to the laboratory structure. The results demonstrated the ability of the proposed technique to detect and localize damage directly from the ambient vibration response of the structure. All 1D CNNs trained in this study had a shallow structure (two CNN layers with only 4 neurons followed by two MLP layers with 5 neurons).

It was noticed that the process of generating the data required to train the 1D CNNs in [48,50,86] requires a large number of measurement sessions especially for a large civil structure. Therefore, Avci et al. in [51] and then Abdeljaber et al. in [52] developed a novel approach based on 1D CNNs, which require significantly less effort and labeled data for training. This approach was successfully tested over the data provided under the Experimental Phase II of the SHM Benchmark Problem [87].

3.4. Other applications

Bearings play an important role in continuous functioning of the rotating machines by being a direct interface between stationary internal supports and rotating components. Not only local bearings but also all types of rotating members affect the overall dynamic behavior, running accuracy, reliability and service life of the global machine structure. As the bearings are continuously exposed to short- and long-term damage during operation; aging is inevitable for these elements. Not only the wearing and tear out, but also due to the lack in handling, repair and maintenance practices, more bearings continue to fail, reducing efficiency and reliability of the entire production line; carrying the potential to cause catastrophic failures of machine parts. Early detection of bearing faults by real-time condition assessment via embedded sensors has the potential to enable replacement of the bearing parts, instead of expensive replacements of the entire machine group. Various studies are available in the literature on diagnosis, prognosis; defect and fault detection; and condition monitoring of bearings and bearing parts in rotating machinery. Different techniques, such as acoustic emission, infra-red thermography, oil analysis have been used to determine the degradation of bearings. While these methods reveal the existence of defects inside bearings; they are not capable of localizing the defect (e.g., the rolling element, the cage, outer or inner race). Among the available approaches, vibration-based fault detection is standing out to be the most effective and reliable approach in revealing, locating and quantifying the damage on the bearing elements. Among the vibration-based techniques, ML based approaches have been predominantly emphasized and used more often for fault diagnosis of rotational machine parts. These methods are predominantly in need of damage-sensitive feature extraction from the recorded signals to be able to train a classifier for condition assessment of the bearing. Most of the available methods carry certain drawbacks and limitations. Due to the increasing complexity of mechanical elements and loading mechanisms, the degradation pattern can result in an enlargement of the damaged area and also in multiplication of number and location of defective structural parts. As a matter of fact, as the number of cracks in defective teeth gears [88] or number of spalls in defective bearings increase, various extracted

features can lose their capability to determine the internal degradation as the number of simultaneous defects increases. In a broader sense, the main drawback of the early ML based methods is the fact that they are highly dependent on hand-crafted features. Such features might be sub-optimal, which means that they cannot accurately characterize the measured vibration signal. As such, when a classifier is trained based on sub-optimal features, it tends to result in unsatisfactory classification performance, and therefore unreliable fault diagnosis results [89].

From this perspective, it is crucial to detect the anomaly as soon as it appears so as to avoid large-scale damages or even worse, fatal outcomes such as electric discharges or potential explosions of machines. Numerous studies based on ML paradigms have been proposed in this domain with varying performance levels. This simply indicates how important the choice of the right features for the characterization of the monitored electric signals (e.g. current or voltage). It is a well-known fact that those fixed and handcrafted set of features cannot effectively characterize any possible electric signal and hence for those cases where their discrimination suffers, the detection performance will deteriorate regardless from the classifier used. This is why they are unable to form a generic solution that can be utilized for any electric waveform or data. Similar to other applications, 1D CNNs have the unique capability to optimize both feature extraction and classification in a single learning body and naturally, the three recent studies [53,56,57], have shown that a real-time monitoring and instantaneous anomaly detection can be accomplished with a state-of-the-art accuracy level. In [53] a novel method based on compact 1D CNNs was proposed to detect a potential motor anomaly due to the bearing faults. Bearing faults are mechanical defects that cause slight variations at certain frequencies in the motor-current waveform. 1D CNNs have been proven to detect anomalies in real-time close to 100% accuracy thanks to the layered sub-band decomposition performed in their hidden CNN-layers.

Another recent 1D CNN application is on high-power multilevel converters that have been utilized extensively for efficient power conversion. The modular multilevel converters (MMC) are arguably the most efficient and feasible multilevel converter topology for medium power to high power applications. The MMC serves as a controllable voltage source with a high number of possible discrete voltage steps while the multilevel topology prevents potential major harmonic content generation. In comparison with other multilevel converter topologies, the predominant features of the MMCs are modularity and scalability to meet any voltage level requirements and performance efficiency in high-voltage applications. MMC is composed of many identical controlled voltage cells. Each cell can have one or more switches and a switch failure may occur in anyone of these cells. The steady-state normal and fault behavior of a cell voltage vary significantly based on the changes in the load current and the fault timing, which makes it difficult to detect and identify such faults in a fast manner. Yet, safety and reliability has become the most important challenges for MMCs, which may encapsulate many power switching devices, each of which may be considered as a potential failure site. For instance, an open-circuit fault in a cell will distort the output voltage and current, which will cause an uncontrolled variation of the floating capacitor voltages and cause the disruption of operation and even a potential failure of the MMC. Even though there are numerous studies for anomaly detection in MMC circuits, many of them contain limitations and drawbacks which may hinder the practical use of them. For example, some studies proposed to put sensor to each cell which may be neither feasible due to the high cost nor reliable since a sensor may fail too. Some other studies required manual feedback and human interaction. Most of them suffer from high computational complexity which hinders their utilization in real-time. The frontier study in [54] where a compact 1D CNN was used first time in the core of the system monitors the cell capacitor voltages and the differential current to detect an open-circuit anomaly almost instantaneously, with a very high accuracy in fault detection and identification (e.g. practically 100%) and excellent reliability and robustness against variations of MMC parameters and fault time. Moreover, the system shows low computational complexity that allows real-time monitoring, and low time delay for fault detection and identification (e.g., <0.1 s). Besides all, this method can easily scale up to very large scale MMCs with hundreds of cells and has the internal capability to detect the multiple faults. Such massive MMC circuits with hundreds or even thousands of SMs can be broken down to group of SMs with a practical size (e.g., 8 SMs) each of which can be monitored by a standalone and dedicated 1D CNN that can detect and localize any fault on that group -if and when occurs. Therefore, a group of identically trained 1D CNNs can monitor the entire MCM circuitry “in parallel” and if anyone detects a fault, the corresponding action (e.g. shutting of the source power) can immediately be taken. This is a straightforward expectation because the proposed 1D CNN detector has already shown the ability to distinguish the pattern of the real fault occurring on a particular switch from the other “distorted” patterns belonging to other switches functioning normally.

4. Computational complexity analysis of 1D-CNNs

In order to analyze the computational complexity for both FP and BP process, we shall first compute the total number of operations at each 1D CNN layer (ignoring the sub-sampling that has a negligible computational cost) and then cumulate them to find the overall computational complexity. During the FP, at a CNN layer, l , the number of connections to the previous layer is, $N^{l-1}N^l$, the number of connections to the previous layer is, $N^{l-1}N^l$, there is an individual linear convolution performed, which is a linear weighted sum. Let s^{l-1} and w^{l-1} be the vector sizes of the previous layer output, s_k^{l-1} , and kernel (weight), w_{ki}^{l-1} , respectively. Ignoring the boundary conditions, a linear convolution consists of $s^{l-1}(w^{l-1})^2$ multiplications and s^{l-1} additions from a single connection. Ignoring the bias addition, the total number of multiplications and additions in the layer l will therefore be:

$$\begin{aligned} N(mul)^l &= N^{l-1} N^l s^{l-1} (w^{l-1})^2, \\ N(add)^l &= N^{l-1} N^l s^{l-1} \end{aligned} \quad (11)$$

So, during FP the total number of multiplications and additions, $T(mul)$, and $T(add)$, on a L CNN layers will be,

$$\begin{aligned} T_{FP}(mul) &= \sum_{l=1}^L N^{l-1} N^l s^{l-1} (w^{l-1})^2, \\ T_{FP}(add) &= \sum_{l=1}^L N^{l-1} N^l s^{l-1} \end{aligned} \quad (12)$$

Obviously, $T(add)$ is insignificant compared to $T(mul)$.

During the BP, there are two convolutions performed as expressed in Eqs. (8) and (9). In Eq. (8) a linear convolution between the delta error in the next layer, Δ_i^{l+1} , and the reversed kernel, $rev(w_{ik}^l)$, in the current layer, l . Let x^l be the size of both the input, x_i^l , and also its delta error, Δ_i^l , vectors of the i^{th} neuron. The number of connections between the two layers is, $N^{l+1} N^l$ and at each connection, the linear convolution in Eq. (8) consists of $x^{l+1} (w^l)^2$ multiplications and x^{l+1} additions. So, again ignoring the boundary conditions, during a BP iteration, the total number of multiplications and additions due to the first convolution will, therefore, be:

$$\begin{aligned} T_{BP}^1(mul) &= \sum_{l=0}^{L-1} N^{l+1} N^l x^{l+1} (w^l)^2, \\ T_{BP}^1(add) &= \sum_{l=0}^{L-1} N^{l+1} N^l x^{l+1} \end{aligned} \quad (13)$$

The second convolution in Eq. (9) is between the current layer output, s^l , and next layer delta error, Δ_i^{l+1} where $w^l = x^{l+1} - s^l$. For each connection, the number of additions and multiplications will be, w^l and $w^l (x^{l+1})^2$, respectively. During a BP iteration, the total number of multiplications and additions due to the second convolution will, therefore, be:

$$\begin{aligned} T_{BP}^2(mul) &= \sum_{l=0}^{L-1} N_{l+1} N_l w_l (x_{l+1})^2, \\ T_{BP}^2(add) &= \sum_{l=0}^{L-1} N_{l+1} N_l w_l \end{aligned} \quad (14)$$

So at each BP iteration, the total number of multiplications and additions will be, $(T_{FP}(mul) + T_{BP}^1(mul) + T_{BP}^2(mul))$ and $(T_{FP}(add) + T_{BP}^1(add) + T_{BP}^2(add))$, respectively. Obviously, the latter is insignificant compared to former especially when the kernel size is high. Moreover, both operation complexities are proportional to the total number of connections between two consecutive layers, which are the multiplication of the number of neurons at each layer. Finally, the computational complexity analysis of MLPs is well known (e.g., see [90]) and it is quite negligible in the current implementation since only a scalar (weight) multiplication and an addition are performed for each connection.

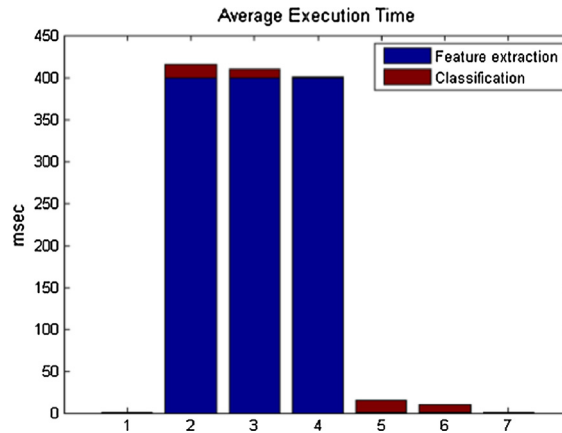


Fig. 9. For the motor fault detection, the average execution times (in msec) of: (1) the 1D CNN method in [53], (2–7) 6 competing algorithms from [91–94].

Without any exception, in all aforementioned 1D CNN applications, a minimal computational complexity is achieved against the competing (conventional) methods due to the aforementioned reasons. For example in the application of [50], using an ordinary computer, when the performance of the 1D CNN was tested, a fault detection speed of 45-times faster than real-time speed was achieved. As another example, Fig. 9 presents the average classification times of the 1D CNN and the 6 competing methods for the application of motor fault detection where the competing methods are from [91–94].

5. Conclusions

Since the introduction of the first “artificial neuron” model by McCulloch-Pitts [3] in 1943, the era of ML has experienced many ups and downs in different stages of the history. Nevertheless, CNNs as its final product are attracting the utmost attention worldwide and influencing almost all aspects of the modern life. On the other hand, deep CNNs alone can have an equal or even better learning ability than humans for the complex patterns or objects in massive size data repositories. Empowered by these, more and more Artificial Intelligence (AI) products are emerging every day, which will soon replace humans for many basic tasks such as driving, assisting, transportation, handling or load carrying, etc. It has already become apparent that AI will further assist or perhaps even replace humans on those complex tasks that require a high level of expertise and training, such as medical operations, health monitoring and diagnosis, taxonomy and even higher education.

1D CNNs are the recent variants of conventional (2D) CNNs. Although they were introduced only a few years ago, recent studies have revealed that with a proper systematic approach, compact 1D CNNs can surpass all the traditional and conventional approaches. In this article, we draw the focus especially on those compact 1D CNNs and present a comprehensive survey on their engineering applications. Compact 1D CNNs can promise a sole advantage of being applicable to those applications where the labeled data for training is scarce and a low-cost, real-time implementation is desired. In such applications, it is obvious that a deep 2D CNN may not be feasible at all due to the scarcity of the training data and the high complexity that eventually violates the real-time constraint. On top of this, the conventional 2D CNNs can only process 2D signals; hence this enforces an extra 1D to 2D transformation following with a windowing (framing) operation, both of which cost additional time and resources. In many applications covered in this article, it has been shown that 1D CNNs are relatively easier to train and offer the minimal computational complexity while achieving state-of-the-art performance levels. They are especially suitable for mobile or hand-held devices with limited computation power and battery life. This is why they are attracting attention with an increasing pace; for instance, the 1D CNN publications, [46] and [50] have immediately become the most-popular and most-cited articles in their journals. The 1D CNN software used in these studies is now publicly shared in [63].

The main limitation or the drawback of 1D CNNs is actually common for conventional CNNs and ANNs in general: They are homogenous (same neuron type in the entire network) and based solely on *linear-neuron* model from 1950s. A recent article [95] from an expert Neuroscientists pointed out this fact as follows: “... *But here’s the thing: for all their similarities to the human brain, artificial deep neural nets are highly reductive models of the seemingly chaotic electro-chemical transmissions that populate every synapse of our own heads. With the big data era in neuroscience upon us, in which we can tease out the delicate wiring and diverse neuronal types (and non-neuron brain cells) that contribute to cognition, current deep learning models seem terribly over-simplistic.*” There are very recent attempts to address this deficiency of “modern-age” deep or compact ANNs. For instance, in studies [7,8], the first generalized neuron and network models, the so-called Generalized Operational Perceptrons (GOPs), have recently been proposed. GOPs can use any neuron model, linear or non-linear while having a heterogeneous network structure just like the human nervous system. Further in, [98–100] GOPs were further improved to obtain other desired features such as neuron-level heterogeneity and “memory” capability. However, it was not until very recently that the first CNN-like network without those aforementioned limitations has been proposed in [96]. This new-generation network is called Operational Neural Networks (ONNs). ONNs can be heterogeneous and encapsulate neurons with any set of operators, linear or non-linear, to boost diversity and to learn highly complex and multi-modal functions or spaces with minimal network complexity and training data. These recent studies have shown that both GOPs and ONNs can indeed achieve significantly superior learning capabilities than the conventional MLPs and CNNs [7,8,98–103]. Particularly, they exhibited an elegant learning performance over those highly complex and challenging problems which defy the conventional MLPs and CNNs. So, we can foresee that the emerging (1D) ONNs may soon replace 1D CNNs providing even more compact and efficient solutions especially for those 1D signal repositories with highly complex and diverse patterns.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Qatar National Research Fund (QNRF) through the ongoing project under Grant NPRP11S-0108-180228. Open Access funding provided by the Qatar National Library.

Appendix A

Real-time electrocardiogram (ECG) monitoring

Despite numerous methods have been proposed for generic ECG classification, high inter-patient variations of ECG signals make a general-purpose modeling and pattern learning infeasible. In other words, a single classifier trained over the ECG signals of the other individuals may not classify accurately the ECG signal of a person. Fig. 10 shows some typical ECG beats where different subjects in the benchmark MIT-BIH arrhythmia dataset have entirely different normal (N) beats, which may, however, show high level of morphological similarities to other subject's abnormal beats (e.g. arrows in the figure below) and vice versa. Obviously, a single classifier will inevitably confuse the patterns of normal/abnormal beats since the same pattern can exist in different beat types from different patients. In order to address this drawback, the first 1D CNN application was on ECG beat identification [45,46] where a "patient-specific" solution was proposed, i.e., for each arrhythmia patient a dedicated compact 1D CNN was trained by using the patient-specific training data as illustrated in Fig. 11.

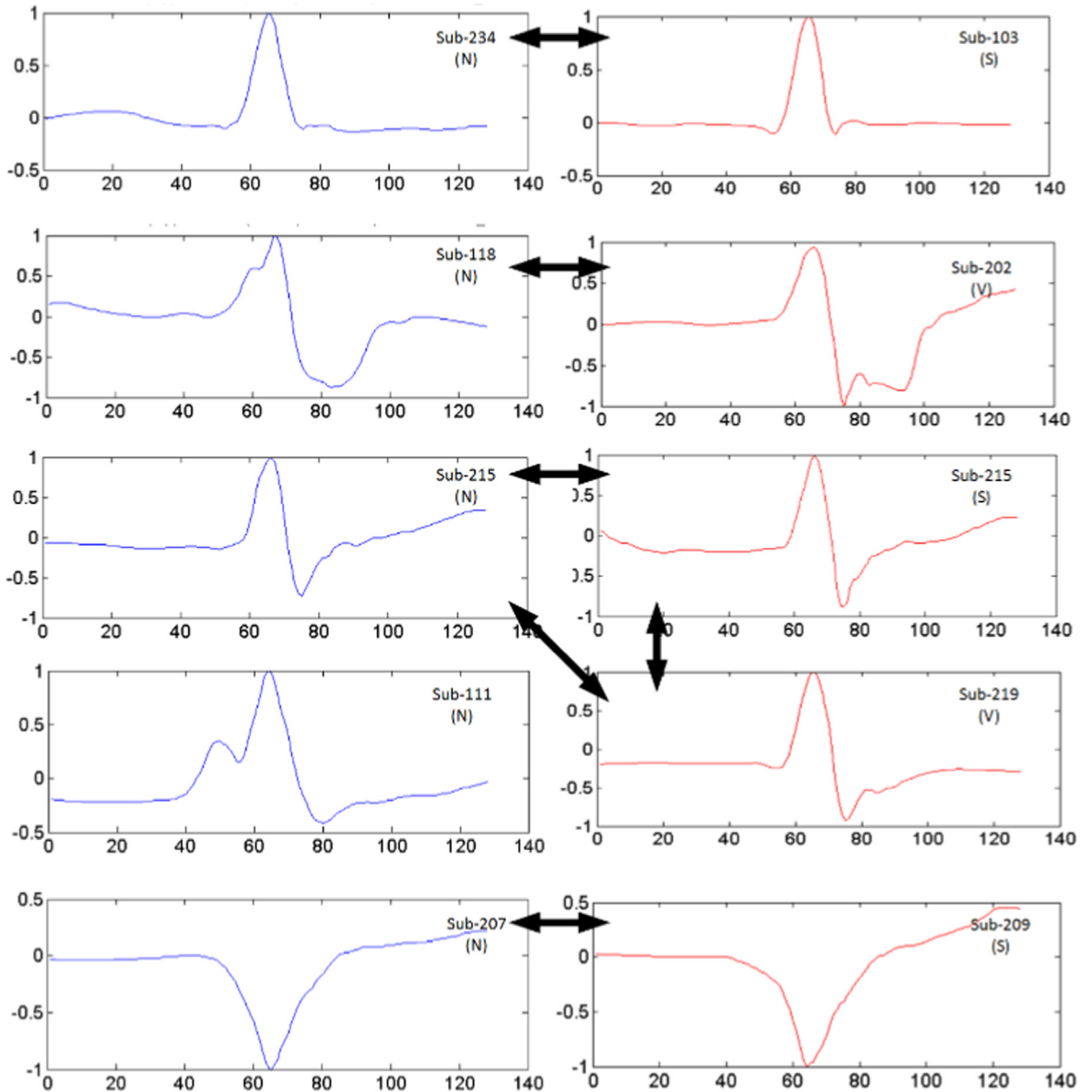


Fig. 10. Normal (N) vs. Abnormal (S and V) beats from different subjects in MIT-BIH dataset. Arrows indicate a high morphological similarity between beats from different classes [47].

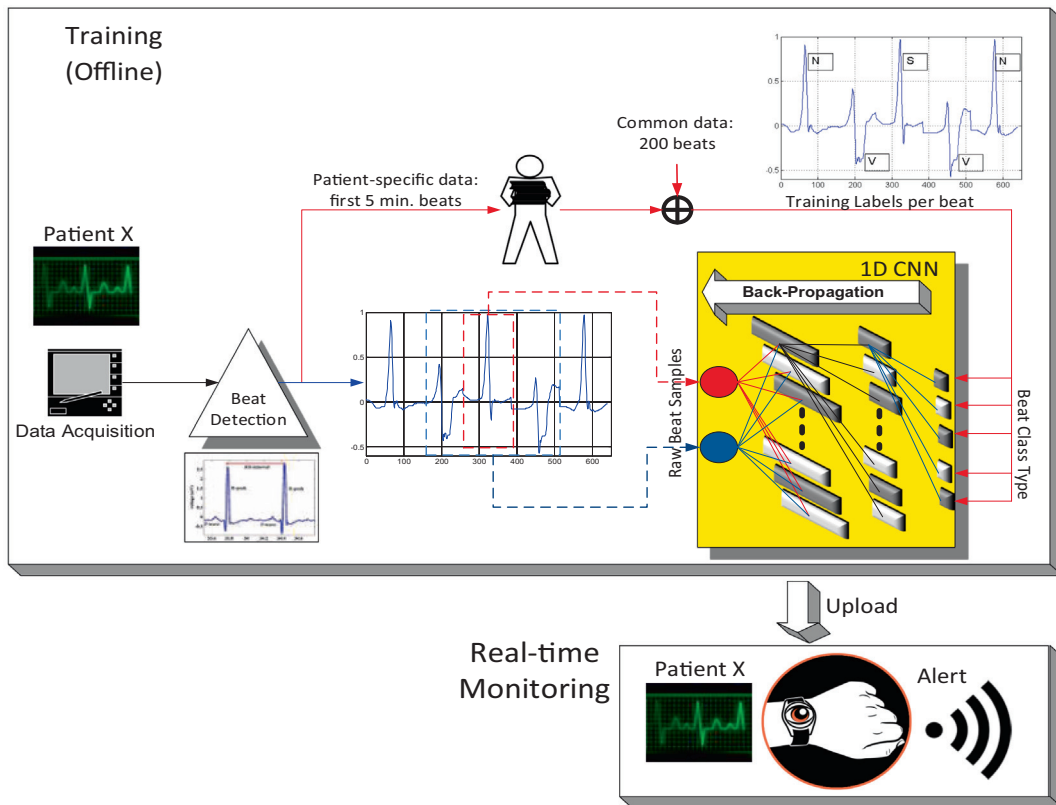


Fig. 11. Overview of the arrhythmia detection and identification system proposed in [45,46].

The purpose of [45,46] is to identify each ECG beat into one of the five classes: N (beats originating in the sinus mode), S (supraventricular ectopic beats), V (ventricular ectopic beats), F (fusion beats), and Q (unclassifiable beats). In these studies, ECG records from the benchmark MIT/BIH arrhythmia database [25] were used for both training and performance evaluation. There are total of 48 records in this benchmark database and each record has two-channel ECG signal for 30-min duration selected from 24-h recordings of 47 individuals. A total of 83,648 beats from all 44 records were used as test patterns for performance evaluation. The beats are represented in both 64 and 128 samples centered around the R-peak. The configuration of the 1D CNN used in all experiments has 3 hidden convolutional layers and 2 dense layers. The 1D CNNs have 32 and 16 neurons on the first and second hidden convolutional layers and 10 neurons on the hidden dense layer. The output layer size is 5 which is the number of beat classes and the input (CNN) layer size is either 2 (*base*) or 4 (*extended*) according to the choice of raw data representation. For 64 and 128 sample beat representations, the kernel sizes are set to 9 and 15, and the sub-sampling factors are set to 4 and 6, respectively. For all experiments a shallow training is employed: the maximum number of BP iterations is set to 50 and another stopping criterion is the minimum train classification error level that is set to 3% to prevent over-fitting. Therefore, the training will terminate if either of the criteria is met. Initially the learning factor, ϵ , is set as 10^{-3} and global learning rate adaptation is performed during each BP iteration, as follows: if the train MSE decreases in the current iteration ϵ is slightly increased by 5%; otherwise, reduced by 30%, for the next epoch. The proposed method in [46] has achieved the highest average accuracies (99% for Ventricular Ectopic Beats (VEB) and 97.6% for Supraventricular Ectopic Beats (SVEB)) on arrhythmia detection with the minimal computational complexity.

The patient-specific ECG classification methods in the past and the one proposed in [45,46] have achieved the state-of-the-art performances. However, such methods require a priori knowledge (labels) of both normal and abnormal beats of the patient. Therefore, the Cardiologist's labelling is a strict requirement of all such methods and that is why they can only be applied to cardiac patients with known arrhythmia in order to classify their "future" ECG records in an automatic way. In brief, they are mere diagnostic tools only for cardiac patients that aim to ease the burden of the MDs and cardiologists to classify long ECG records. For the case of "healthy" people with no history of arrhythmia, obviously, none can be used as a health-health monitoring and advance warning system due to simple fact that no "personal" abnormal ECG data yet exist. To address this crucial problem, A recent study [47] has presented a real-time and fully automatic solution for *personalized* cardiac-health monitoring and early detection of cardiac arrhythmias from the electrocardiogram (ECG) data. The proposed solution involves a systematic approach to:

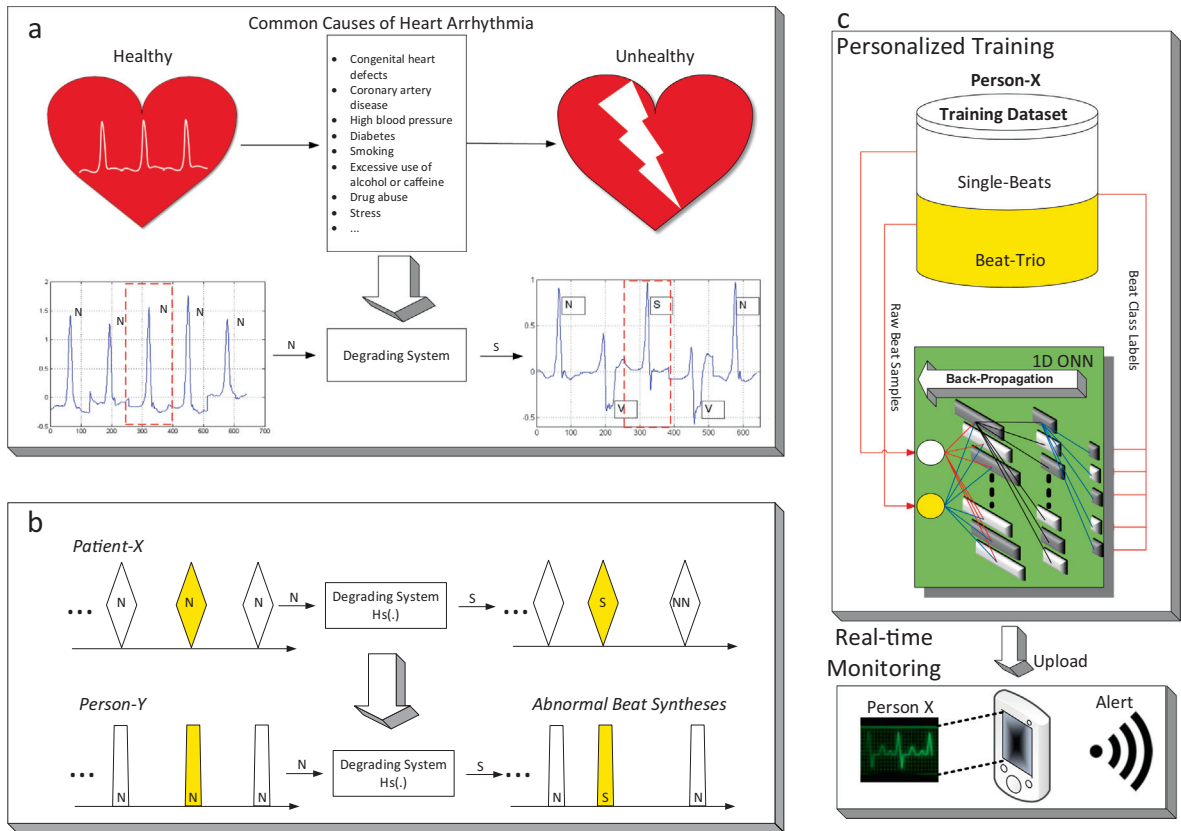


Fig. 12. Taken from [47].

- 1) model the common causes of cardiac arrhythmias using a set of ABS filters,
- 2) synthesize all potential abnormal beats for a “healthy person” from his/her normal ECG beats,
- 3) form the personalized training dataset for the healthy person using the synthesized abnormal (arrhythmic) beats and real normal beats,
- 4) train a personal classifier, a 1D CNN, over the personalized training dataset,
- 5) use the personal classifier (the trained 1D CNN) in real-time to detect any real arrhythmia –if and when occurs.

This study has become the first attempt to propose a personalized early detection of ECG anomalies and cardiac health monitoring. In the absence of real abnormal beats, this becomes a far more challenging problem than the patient-specific ECG beat classification. The key accomplishment in this work is that the common causes of cardiac arrhythmias are modeled by a set of filters and then they are used to synthesize appropriate potential abnormal beats of a healthy person as illustrated in Fig. 12. Upon learning the healthy person’s (real) normal beats and potential (synthesized) abnormal beats, the proposed system with 1D CNNs can then be used to detect any abnormal beat which may occur during monitoring.

- a) Modeling common causes of cardiac arrhythmia in the signal domain by a degrading system.
- b) A symbolic illustration of an abnormal S beat synthesis for *Person-Y* using the degrading system designed from the ECG data of *Patient-X*.
- c) Illustration of the overall system, where a dedicated CNN is trained by Back-Propagation over the training dataset created for *Person-X* (top). Once the 1D CNN is trained, it can then be used as a continuous cardiac health monitoring and advance warning system (bottom) for *Person-X*.

It is worth mentioning here that in order to properly model the common cause of the arrhythmia, there is no need to identify it precisely, which may not be even feasible. However, it is of paramount importance to capture its degradation (the relation between the normal to abnormal beat) in the signal domain so as to use this information (model) to synthesize personal abnormal beats for a healthy person using his/her normal beat. This is why the illustrative example in Fig. 10 is quite useful: the relation between a normal (N) and supraventricular ectopic (S) beat was first modelled over the ECG signal of *Patient-X* by an ABS filter, which is then used to synthesize a S beat for the (healthy) *Person-Y*. The reason for the S beat occurrence in

Patient-X's ECG signal may be drug use, excess caffeine, stress or any other but there is no need to identify that in a precise way to properly model it with an ABS filter.

Over more than 63,000 ECG beats and 34 subjects, the study has demonstrated that the proposed approach achieved a very high detection accuracy with a very low false alarm rate. For the cardiac patients, it has been shown that the proposed method can detect the real abnormal beats with a high accuracy without using the Cardiologist labels for abnormal beats. This is why the method is fully automatic and does not require any manual feedback, tuning or intervention. Furthermore, for the healthy subjects it has been shown that the proposed method is quite reliable, producing insignificant number of false alarms. Without using the real abnormal beats in training, the proposed method has achieved accuracy level, $Acc = 80.1\%$ and false-alarm rate, $FAR = 0.43\%$. The average probability of missing the first abnormal beat, therefore, is 0.199. In addition, the average probability of missing all three consecutive abnormal beats is around 0.0079. As a result, detecting one or more abnormal beat(s) among the first three occurrences is highly probable (greater than 99.2%).

References

- [1] A. Cichoki, R. Unbehauen, *Neural Networks for Optimization and Signal Processing*, thirrd ed., 1994.
- [2] S.O. Haykin, *Neural Networks and Learning Machines*, 2008. doi:978-0131471399.
- [3] S.M. Warren, P. Walter, A logical calculus of the ideas immanent in nervous activity, *Bull. Math. Biophys.* 5 (1943) 115–133.
- [4] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychol. Rev.* 65 (1958) 386.
- [5] J.P. Resop, A Comparison of Artificial Neural Networks and Statistical Regression with Biological Resources Applications, 2006.
- [6] T. Ince, S. Kiranyaz, J. Pulkkinen, M. Gabbouj, Evaluation of global and local training techniques over feed-forward neural network architecture spaces for computer-aided medical diagnosis, *Expert Syst. Appl.* 37 (2010) 8450–8461, <https://doi.org/10.1016/j.eswa.2010.05.033>.
- [7] S. Kiranyaz, T. Ince, A. Iosifidis, Moncef Gabbouj, Progressive operational perceptrons, *Neurocomputing* (2016), <https://doi.org/10.1016/j.neucom.2016.10.044>.
- [8] S. Kiranyaz, T. Ince, A. Iosifidis, M. Gabbouj, Generalized model of biological neural networks: progressive operational perceptrons, in: *Proc. Int. Jt. Conf. Neural Networks. 2017–May (2017)* 2477–2485. <https://doi.org/10.1109/IJCNN.2017.7966157>.
- [9] T.W. Rauber, K. Berns, Kernel multilayer perceptron, in: *Proc. - 24th SIBGRAPI Conf. Graph. Patterns Images*, 2011 pp. 337–343. <https://doi.org/10.1109/SIBGRAPI.2011.21>.
- [10] H. Ogai, B. Bhattacharya, Pipe Inspection Robots for Structural Health and Condition Monitoring, 2018. <https://doi.org/10.1007/978-81-322-3751-8>.
- [11] M.Y. Mashor, Hybrid multilayered perceptron networks, *Int. J. Syst. Sci.* 31 (2000) 771–785, <https://doi.org/10.1080/00207720050030815>.
- [12] S. Kiranyaz, T. Ince, A. Yildirim, M. Gabbouj, Evolutionary artificial neural networks by multi-dimensional particle swarm optimization, *Neural Networks* 22 (2009) 1448–1462, <https://doi.org/10.1016/j.neunet.2009.05.013>.
- [13] T. Ince, S. Kiranyaz, M. Gabbouj, A generic and robust system for automated patient-specific classification of ECG signals, *IEEE Trans. Biomed. Eng.* 56 (2009) 1415–1426, <https://doi.org/10.1109/TBME.2009.2013934>.
- [14] S. Kiranyaz, T. Ince, M. Gabbouj, *Multidimensional Particle Swarm Optimization for Machine Learning and Pattern Recognition*, Springer, 2014.
- [15] S.A. Mohseni, A.H. Tan, Optimization of neural networks using variable structure systems, *IEEE Trans. Syst. Man, Cybern. Part B Cybern.* 42 (2012) 1645–1653, <https://doi.org/10.1109/TSMCB.2012.2197610>.
- [16] D.H. Hubel, T.N. Wiesel, Receptive fields of single neurones in the cat's striate cortex, *J. Physiol.* 148 (1959) 574–591.
- [17] D.H. Hubel, Single unit activity in lateral geniculate body and optic tract of unrestrained cats, *J. Physiol.* 150 (1960) 91–104, <https://doi.org/10.1113/jphysiol.1960.sp006375>.
- [18] D.H. Hubel, T.N. Wiesel, Receptive fields, binocular interaction and functional architecture in the cat's visual cortex, *J. Physiol.* 160 (1962) 106–154.
- [19] D.H. Hubel, T.N. Wiesel, Receptive fields of cells in striate cortex of very young, visually inexperienced kittens, *J. Neurophysiol.* 26 (1963) 994–1002.
- [20] D.H. Hubel, T.N. Wiesel, Receptive fields and functional architecture of monkey striate cortex, *J. Physiol.* 195 (1968) 215–243.
- [21] K. Fukushima, S. Miyake, Neocognitron: a new algorithm for pattern recognition tolerant of deformations and shifts in position, *Pattern Recognit.* 15 (1982) 455–469, [https://doi.org/10.1016/0031-3203\(82\)90024-3](https://doi.org/10.1016/0031-3203(82)90024-3).
- [22] D.E. Rumelhart, G. Hinton, R.J. Williams, Learning representations by back-propagation errors, *Nature* 323 (1986) 533–536.
- [23] Y. LeCun, B. Boser, J.S. Denker, R.E. Howard, W. Hubbard, L.D. Jackel, Handwritten digit recognition with a back-propagation network, *Adv. Neural Inf. Process. Syst.* (1990) 396–404, <https://doi.org/10.1111/dsu.12130>.
- [24] Y. LeCun, *Gradient Based Learning Applied To Document Recognition*, 1998, pp. 1–46.
- [25] L. Yann, C. Corinna, B. Chris, *MNIST Handwritten Digit Database*, New York Univ, 2018.
- [26] A. Krizhevsky, I. Sutskever, G. Hinton, ImageNet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012) 1097–1105.
- [27] J. Deng, W. Dong, R. Socher, L.-J. Li, Kai Li, Li Fei-Fei, ImageNet: a large-scale hierarchical image database, in: *2009 IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255. <https://doi.org/10.1109/CVPR.2009.5206848>.
- [28] L. Juan, A comparison of SIFT, PCA-SIFT and SURF, *Int. J. Image Process* 3 (2010) 143–152.
- [29] T. Ojala, M. Pietikainen, Multiresolution gray-scale and rotation invariant texture classification with local binary patterns, *IEEE Trans. Pattern Anal. Mach. Intell.* 24 (2005) 971–987.
- [30] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, *J. Mach. Learn. Res.* 1 (2004) 11, <https://doi.org/10.1016/j.micromeso.2003.09.025>.
- [31] M.D. Zeiler, R. Fergus, Visualizing and understanding convolutional neural networks, *Lect. Notes Comput. Sci. (Including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, 8689 LNCS (2014) 818–833. https://doi.org/10.1007/978-3-319-10590-1_53.
- [32] O. Abdel-Hamid, A.R. Mohamed, H. Jiang, G. Penn, Applying convolutional neural networks concepts to hybrid NN-HMM model for speech recognition, in: *ICASSP, IEEE Int. Conf. Acoust. Speech Signal Process. - Proc.*, 2012, pp. 4277–4280. <https://doi.org/10.1109/ICASSP.2012.6288864>.
- [33] Y. Kim, Convolutional neural networks for sentence classification, in: *Proc. 2014 Conf. Empir. Methods Nat. Lang. Process.*, 2014, pp. 1746–1751.
- [34] C. Lu, Z. Wang, B. Zhou, Intelligent fault diagnosis of rolling bearing using hierarchical convolutional network based health state classification, *Adv. Eng. Inform.* 32 (2017) 139–151, <https://doi.org/10.1016/j.aei.2017.02.005>.
- [35] X. Ding, Q. He, Energy-fluctuated multiscale feature learning with deep ConvNet for Intelligent spindle bearing fault diagnosis, *IEEE Trans. Instrum. Meas.* 66 (2017) 1926–1935, <https://doi.org/10.1109/TIM.2017.2674738>.
- [36] J. Guo, L. Chen, C. Shen, Hierarchical adaptive deep convolution neural network and its application to bearing fault diagnosis, *Meas. J. Int. Meas. Confed.* 93 (2016) 490–502, <https://doi.org/10.1016/j.measurement.2016.07.054>.
- [37] O. Janssens, V. Slavkovic, B. Vervisch, K. Stockman, M. Locufier, S. Verstockt, R. Van de Walle, S. Van Hoecke, Convolutional neural network based fault detection for rotating machinery, *J. Sound Vib.* 377 (2016) 331–345, <https://doi.org/10.1016/j.jsv.2016.05.027>.
- [38] D. Hoang, H. Kang, Convolutional Neural Network Based Bearing Fault Diagnosis, 10362 (2017) 105–111. <https://doi.org/10.1007/978-3-319-63312-1>.
- [39] Z. Wei, P. Gao, L. Chuanhao, Bearings Fault Diagnosis Based on Convolutional Neural Networks with 2-D Representation of Vibration Signals as Input, 13001 (2017) 1–5.

- [40] D.K. Appana, W. Ahmad, J.-M. Kim, Speed invariant bearing fault characterization using convolutional, Neural Networks (2017) 189–198, https://doi.org/10.1007/978-3-319-69456-6_16.
- [41] S. Li, G. Liu, X. Tang, J. Lu, J. Hu, An ensemble deep convolutional neural network model with improved D-S evidence fusion for bearing fault diagnosis, *Sensors* 17 (2017) 1729, <https://doi.org/10.3390/s17081729>.
- [42] K.B. Lee, S. Cheon, C.O. Kim, L. Dalfino, F. Puntillo, A. Mosca, R. Monno, M. Luigia, S. Coppolecchia, G. Miragliotta, F. Bruno, D. Verstraete, M. Engineering, M. Engineering, M.F. Guo, X.D. Zeng, D.Y. Chen, N.C. Yang, An adaptive deep convolutional neural network for rolling bearing fault diagnosis, *Hindawi Shock Vib.* 30 (2017) 1–29, <https://doi.org/10.1109/TSM.2017.2676245>.
- [43] J. Ruiz, J. Pérez, J. Blázquez, Arrhythmia detection using convolutional neural models, in: *Int. Symp. Distrib. Comput. Artif. Intell.*, 2018.
- [44] M. Zihlmann, D. Perekestenko, M. Tschannen, *Convolutional recurrent neural networks for electrocardiogram classification*, *Computing* (2017).
- [45] S. Kiranyaz, T. Ince, R. Hamila, M. Gabbouj, Convolutional Neural Networks for patient-specific ECG classification, in: *Proc. Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. EMBS*, 2015, <https://doi.org/10.1109/EMBC.2015.7318926>.
- [46] S. Kiranyaz, T. Ince, M. Gabbouj, Real-time patient-specific ECG classification by 1-D convolutional neural networks, *IEEE Trans. Biomed. Eng.* 63 (2016) 664–675, <https://doi.org/10.1109/TBME.2015.2468589>.
- [47] S. Kiranyaz, T. Ince, M. Gabbouj, Personalized monitoring and advance warning system for cardiac arrhythmias, *Sci. Rep.* 7 (2017), <https://doi.org/10.1038/s41598-017-09544-z>.
- [48] O. Avci, O. Abdeljaber, S. Kiranyaz, M. Hussein, D.J. Inman, *Wireless and real-time structural damage detection: a novel decentralized method for wireless sensor networks*, *J. Sound Vib.* (2018).
- [49] O. Avci, O. Abdeljaber, S. Kiranyaz, D. Inman, Structural damage detection in real time: implementation of 1D convolutional neural networks for SHM applications, in: C. Niezrecki (Ed.), *Struct. Heal. Monit. Damage Detect.* Vol. 7 *Proc. 35th IMAC, A Conf. Expo. Struct. Dyn.* 2017, Springer International Publishing, Cham, 2017, pp. 49–54, https://doi.org/10.1007/978-3-319-54109-9_6.
- [50] O. Abdeljaber, O. Avci, S. Kiranyaz, M. Gabbouj, D.J. Inman, Real-time vibration-based structural damage detection using one-dimensional convolutional neural networks, *J. Sound Vib.* 388 (2017), <https://doi.org/10.1016/j.jsv.2016.10.043>.
- [51] O. Avci, O. Abdeljaber, S. Kiranyaz, B. Boashash, H. Sodano, D.J. Inman, Efficiency Validation of One Dimensional Convolutional Neural Networks for Structural Damage Detection Using a SHM Benchmark Data, 2018.
- [52] O. Abdeljaber, O. Avci, M.S. Kiranyaz, B. Boashash, H. Sodano, D.J. Inman, 1-D CNNs for structural damage detection: verification on a structural health monitoring benchmark data, *Neurocomputing* (2017), <https://doi.org/10.1016/j.neucom.2017.09.069>.
- [53] T. Ince, S. Kiranyaz, L. Eren, M. Askar, M. Gabbouj, Real-time motor fault detection by 1-D convolutional neural networks, *IEEE Trans. Ind. Electron.* 63 (2016) 7067–7075, <https://doi.org/10.1109/TIE.2016.2582729>.
- [54] S. Kiranyaz, A. Gastli, L. Ben-Brahim, N. Alemadi, M. Gabbouj, Real-time fault detection and identification for MMC using 1D convolutional neural networks, *IEEE Trans. Ind. Electron.* (2018), <https://doi.org/10.1109/TIE.2018.2833045>.
- [55] O. Abdeljaber, S. Sassi, O. Avci, S. Kiranyaz, I. Abulrahman, M. Gabbouj, Fault detection and severity identification of ball bearings by online condition monitoring, *IEEE Trans. Ind. Electron.* (2018).
- [56] L. Eren, T. Ince, S. Kiranyaz, A generic intelligent bearing fault diagnosis system using compact adaptive 1D CNN classifier, *J. Signal Process. Syst.* 91 (2019) 179–189, <https://doi.org/10.1007/s11265-018-1378-3>.
- [57] L. Eren, Bearing fault detection by one-dimensional convolutional neural networks, *Math. Probl. Eng.* 2017 (2017), <https://doi.org/10.1155/2017/8617315>.
- [58] W. Zhang, C. Li, G. Peng, Y. Chen, Z. Zhang, A deep convolutional neural network with new training methods for bearing fault diagnosis under noisy environment and different working load, *Mech. Syst. Signal Process.* 100 (2018) 439–453, <https://doi.org/10.1016/j.ymssp.2017.06.022>.
- [59] G.E. Hinton, R.R. Salakhutdinov, Reducing the dimensionality of data with neural networks, *Science* (80-) (2006), <https://doi.org/10.1126/science.1127647>.
- [60] U.R. Acharya, H. Fujita, O.S. Lih, Y. Hagiwara, J.H. Tan, M. Adam, Automated detection of arrhythmias using different intervals of tachycardia ECG segments with convolutional neural network, *Inf. Sci. (Ny)* (2017), <https://doi.org/10.1016/j.ins.2017.04.012>.
- [61] Z. Xiong, M. Stiles, J. Zhao, Robust ECG signal classification for detection of atrial fibrillation using a novel neural network, in: *2017 Comput. Cardiol.*, 2017.
- [62] U.R. Acharya, S.L. Oh, Y. Hagiwara, J.H. Tan, M. Adam, A. Gertych, R.S. Tan, A deep convolutional neural network model to classify heartbeats, *Comput. Biol. Med.* (2017), <https://doi.org/10.1016/j.combiomed.2017.08.022>.
- [63] O. Avci, S. Kiranyaz, O. Abdeljaber, Structural Damage Detection (Public Website), 2019, <http://www.structuraldamagedetection.com/>.
- [64] N. Qian, On the momentum term in gradient descent learning algorithms, *Neural Networks* 12 (1999) 145–151, [https://doi.org/10.1016/S0893-6080\(98\)00116-6](https://doi.org/10.1016/S0893-6080(98)00116-6).
- [65] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *COLT 2010 - 23rd Conf. Learn. Theory* (2010) 257–269.
- [66] T. Tieleman, G. Hinton, Lecture 6.5 - RMSProp, *Neural Networks for Machine Learning | Coursera*, (n.d.).
- [67] K. Diederik, J.L. Ba, ADAM: a method for stochastic optimization, *AIP Conf. Proc.* 1631 (2014) 58–62, <https://doi.org/10.1063/1.4902458>.
- [68] S. Ruder, An overview of gradient descent optimization algorithms, 2016.
- [69] H. Jiang, Discriminative training of HMMs for automatic speech recognition: a survey, *Comput. Speech Lang.* 24 (2010) 589–608, <https://doi.org/10.1016/j.csl.2009.08.002>.
- [70] X. He, L. Deng, W. Chou, Discriminative learning in sequential pattern recognition: a unifying review for optimization-oriented speech recognition, *IEEE Signal Process. Mag.* 25 (2008) 14–36, <https://doi.org/10.1109/MSP.2008.926652>.
- [71] L. Deng, X. Li, Machine learning paradigms for speech recognition: an overview, *IEEE Trans. Audio, Speech Lang. Process.* 21 (2013) 1060–1089, <https://doi.org/10.1109/TASL.2013.2244083>.
- [72] G. Hinton, L. Deng, D. Yu, G. Dahl, A.R. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. Sainath, B. Kingsbury, Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups, *IEEE Signal Process. Mag.* 29 (2012) 82–97, <https://doi.org/10.1109/MSP.2012.2205597>.
- [73] O. Abdel-Hamid, A.R. Mohamed, H. Jiang, L. Deng, G. Penn, D. Yu, Convolutional neural networks for speech recognition, *IEEE Trans. Audio, Speech Lang. Process.* 22 (2014) 1533–1545, <https://doi.org/10.1109/TASLP.2014.2339736>.
- [74] H. Lee, L. Yan, P. Pham, A.Y. Ng, Unsupervised feature learning for audio classification using convolutional deep belief networks, in: *Adv. Neural Inf. Process. Syst.* 22 - *Proc. 2009 Conf.*, 2009, pp. 1096–1104.
- [75] D. Hu, K. Chen, Exploring hierarchical speech representations with a deep convolutional neural network, in: *Proc. UKCI'11*, 2011.
- [76] M. Bi, Y. Qian, K. Yu, Very deep convolutional neural networks for LVCSR, in: *Proc. Annu. Conf. Int. Speech Commun. Assoc. INTERSPEECH*. 2015–Janua, 2015, pp. 3259–3263.
- [77] T. Sercu, C. Puhresch, B. Kingsbury, Y. Lecun, Very deep multilingual convolutional neural networks for LVCSR, *ICASSP*, in: *IEEE Int. Conf. Acoust. Speech Signal Process.* - *Proc.* 2016–May, 2016, pp. 4955–4959, <https://doi.org/10.1109/ICASSP.2016.7472620>.
- [78] D. Yu, W. Xiong, J. Droppo, A. Stolcke, G. Ye, J. Li, G. Zweig, Deep convolutional neural networks with layer-wise context expansion and attention, in: *Proc. Annu. Conf. Int. Speech Commun. Assoc. INTERSPEECH*. 08–12–Sept, 2016, pp. 17–21, <https://doi.org/10.21437/Interspeech.2016-251>.
- [79] T. Zhao, Y. Zhao, X. Chen, Time-frequency kernel-based CNN for speech recognition, in: *Proc. Annu. Conf. Int. Speech Commun. Assoc. INTERSPEECH*. 2015–Janua, 2015, pp. 1888–1892.
- [80] Y. Yu, C. Wang, X. Gu, J. Li, A novel deep learning-based method for damage identification of smart building structures, *Struct. Heal. Monit.* 18 (2019) 143–163, <https://doi.org/10.1177/1475921718804132>.

- [81] H. Khodabandehlou, G. Pekcan, M.S. Fadali, Vibration-based structural condition assessment using convolution neural networks, *Struct. Control Heal. Monit.* (2018), <https://doi.org/10.1002/stc.2308>.
- [82] Y.J. Cha, W. Choi, O. Büyüköztürk, Deep learning-based crack damage detection using convolutional neural networks, *Comput. Civ. Infrastruct. Eng.* (2017), <https://doi.org/10.1111/mice.12263>.
- [83] N.S. Gulgeç, M. Takáč, S.N. Pakzad, Structural damage detection using convolutional neural networks, in: *Conf. Proc. Soc. Exp. Mech. Ser.*, 2017, https://doi.org/10.1007/978-3-319-54858-6_33.
- [84] O. Abdeljaber, A. Younis, O. Avci, N. Catbas, M. Gul, O. Celik, H. Zhang, Dynamic testing of a laboratory stadium structure, in: *Geotech. Struct. Eng. Congr. 2016*, 2016, pp. 1719–1728. <https://doi.org/10.1061/9780784479742.147>.
- [85] S. Kiranyaz, O. Avci, O. Abdeljaber, Real-time structural damage detection by convolutional neural networks, US 20190017911A1, 2019.
- [86] O. Avci, O. Abdeljaber, S. Kiranyaz, D.J. Inman, Convolutional neural networks for real-time and wireless damage detection, in: *IMAC XXXVII, Int. Modal Anal. Conf.*, Springer International Publishing, Orlando, FL, USA, 2019.
- [87] S. Dyke, D. Bernal, J. Beck, C. Ventura, Experimental phase II of the structural health monitoring benchmark problem, in: *Proc. 16th ASCE Eng. Mech. Conf.*, 2003, pp. 1–7.
- [88] S. Sassi, B. Badri, M. Thomas, Tracking surface degradation of ball bearings by means of new time domain scalar indicators, *Int. J. COMADEM* 11 (2008) 36–45.
- [89] A. Mohamed, S. Sassi, M.R. Paurobally, Model-based analysis of gears' dynamic behavior in the presence of multiple cracks, *J. Shock Vib.* (2018).
- [90] G. Serpen, Z. Gao, Complexity analysis of multilayer perceptron neural network embedded into a wireless sensor network, *Proc. Comput. Sci.* 36 (2014) 192–197, <https://doi.org/10.1016/j.procs.2014.09.078>.
- [91] M. Blodt, P. Granjon, B. Raison, G. Rostaing, Models for bearing damage detection in induction motors using stator current monitoring, *IEEE Trans. Ind. Electron.* 55 (2008) 1813–1822, <https://doi.org/10.1109/TIE.2008.917108>.
- [92] L. Eren, M.J. Devaney, Bearing damage detection via wavelet packet decomposition of the stator current, *IEEE Trans. Instrum. Meas.* 53 (2004) 431–436, <https://doi.org/10.1109/TIM.2004.823323>.
- [93] L. Eren, A. Karahoca, M.J. Devaney, Neural network based motor bearing fault detection, in: *Conf. Rec. - IEEE Instrum. Meas. Technol. Conf.*, 2004, <https://doi.org/10.1109/IMTC.2004.1351399>.
- [94] G.F. Bin, J.J. Gao, X.J. Li, B.S. Dhillon, Early fault diagnosis of rotating machinery based on wavelet packets - empirical mode decomposition feature extraction and neural network, *Mech. Syst. Signal Process.* 27 (2012) 696–711, <https://doi.org/10.1016/j.ymssp.2011.08.002>.
- [95] S. Fan <https://singularityhub.com/2019/02/19/giving-neural-nets-an-innate-brain-like-structure-could-bolster-deep-learning/>.
- [96] S. Kiranyaz, T. Ince, A. Iosifidis, M. Gabbouj, Operational Neural Networks", *Neural Computing and Applications (Springer-Nature)* (Mar. 2020) 1–24, <https://doi.org/10.1007/s00521-020-04780-3>.
- [97] O. Avci, O. Abdeljaber, S. Kiranyaz, M. Hussein, M. Gabbouj, D. Inman, A Review of Vibration-Based Damage Detection in Civil Structures: From Traditional Methods to Machine Learning and Deep Learning Applications, *Mechanical Systems and Signal Processing* 147 (2021) 107077, <https://doi.org/10.1016/j.ymssp.2020.107077>.
- [98] Dath Tran et al, Heterogeneous Multilayer Generalized Operational Perceptron, *IEEE Transactions on Neural Networks and Learning Systems* (2019) 1–15, <https://doi.org/10.1109/TNNLS.2019.2914082>.
- [99] Dat Tran et al, Progressive Operational Perceptron with Memory, *Neurocomputing* (2019), <https://doi.org/10.1016/j.neucom.2019.10.079>.
- [100] Dat Tran, PyGOP: A Python Library for Generalized Operational Perceptron, *Knowledge-Based Systems* 182 (2019), <https://doi.org/10.1016/j.knosys.2019.06.009>.
- [101] Serkan Kiranyaz et al, Exploiting Heterogeneity in Operational Neural Networks by Synaptic Plasticity, arXiv:2004.11778 (2020), Submitted for publication.
- [102] Serkan Kiranyaz et al, Self-Organized Operational Neural Networks with Generative Neurons, arXiv:2004.11778 (2020), Submitted for publication.
- [103] Junaid Malik et al, FastONN–Python based open-source GPU implementation for Operational Neural Networks, arXiv:2006.02267 (2020), Submitted for publication.